



Бенчмаркинг PostgreSQL Выбор сервера для нагрузки

Модуль 1 - Введение в PostgreSQL и бенчмаркинг

Урок 1 - Знакомство с PostgreSQL

Шаг 01:

Миссия 1

Что такое PostgreSQL и какие его основные особенности?

Подсказка: Это объектно-реляционная система управления базами данных с открытым исходным кодом.

Ответ:

- Система управления базами данных с открытым исходным кодом
- Только реляционная система управления базами данных
- Система для анализа больших данных
- Только объектно-ориентированная СУБД

Шаг 02:

Миссия 2

Какой компонент PostgreSQL отвечает за выполнение запросов?

Подсказка: Этот компонент занимается обработкой SQL-запросов и их планированием.

Ответ:

- Планировщик запросов

- Менеджер памяти
- Процессор транзакций
- Репликационный механизм

🔮 Шаг 03:

Миссия 3

Как можно улучшить производительность PostgreSQL с помощью индексов?

Подсказка: Индексы ускоряют поиск данных по определенным полям.

Ответ:

- **Добавление индексов на часто используемые поля**
- Удаление всех индексов для экономии места
- Использование индексов только для JOIN-запросов
- Создание индексов только для строковых данных

🔮 Шаг 04:

Миссия 4

📖 Задача: Сопоставьте компоненты PostgreSQL с их задачами

Понятия:

Ответ:

- Планировщик запросов
- Менеджер транзакций
- Менеджер памяти
- Репликационный механизм

Задачи:

1. Обрабатывает запросы SQL и строит план выполнения.
2. Отвечает за сохранение данных в журнал транзакций.
3. Управляет использованием оперативной памяти для обработки запросов.
4. Отвечает за создание копий данных для обеспечения отказоустойчивости.

🔮 Подсказка:

Каждый компонент выполняет свою уникальную задачу, связанную с эффективной работой СУБД.

- **Планировщик запросов** — Обрабатывает запросы SQL и строит план выполнения.
- **Менеджер транзакций** — Отвечает за сохранение данных в журнал транзакций.
- **Менеджер памяти** — Управляет использованием оперативной памяти для обработки запросов.
- **Репликационный механизм** — Отвечает за создание копий данных для обеспечения отказоустойчивости.

🔮 Шаг 05:

Миссия 5

Какую команду использовать для мониторинга запросов в PostgreSQL?

Подсказка: Команда, показывающая статистику по выполненным запросам.

Ответ:

- **EXPLAIN ANALYZE**

- SHOW STATISTICS
- VACUUM ANALYZE
- SELECT * FROM pg_stat_activity

Шаг 06:

Миссия 6

Расположите этапы выполнения запроса PostgreSQL в правильном порядке:

1. Парсинг SQL-запроса
2. Определение оптимального плана выполнения
3. Выполнение запроса
4. Передача результата пользователю

Подсказка:

Процесс выполнения запроса в PostgreSQL начинается с его обработки, а завершается отправкой данных пользователю.

Ответ:

- Парсинг SQL-запроса
- Определение оптимального плана выполнения
- Выполнение запроса
- Передача результата пользователю

Шаг 07:

Миссия 7

Какая команда в PostgreSQL используется для анализа производительности запросов?

Подсказка: Команда помогает понять, какие части запроса наиболее затратны по времени.

Ответ:

- **EXPLAIN ANALYZE**
- SHOW STATISTICS
- VACUUM ANALYZE
- SELECT * FROM pg_stat_activity

Шаг 08:

Миссия 8

Что такое репликация в PostgreSQL?

Подсказка: Это механизм, позволяющий синхронизировать данные между несколькими серверами.

Ответ:

- **Синхронизация данных между несколькими серверами**
- Процесс резервного копирования данных
- Сохранение всех транзакций на отдельном сервере
- Процесс очистки базы данных от старых данных

Шаг 09:

Миссия 9

 Задача: Сопоставьте виды репликации в PostgreSQL с их описанием

Виды репликации:

Ответ:

- Синхронная репликация
- Асинхронная репликация

Описания:

1. Данные записываются на все сервера одновременно, гарантируя целостность данных.
2. Данные записываются только на основной сервер, что может привести к потере данных в случае сбоя.

🔮 **Подсказка:**

Обратите внимание на время синхронизации и устойчивость к сбоям.

- **Синхронная репликация** — Данные записываются на все сервера одновременно, гарантируя целостность данных.
- **Асинхронная репликация** — Данные записываются только на основной сервер, что может привести к потере данных в случае сбоя.

🔮 **Шаг 10:**

Миссия 10

Что из следующего следует учитывать при выборе сервера для PostgreSQL?

Подсказка: Основной фокус — это производительность и доступность ресурсов.

Ответ:

- **Процессор, память, диск и сеть**
- Только процессор и память
- Только количество оперативной памяти
- Только скорость диска и сети

🔮 **Шаг 11:**

Миссия 11

Расположите факторы, влияющие на производительность PostgreSQL, в правильном порядке:

1. Процессор
2. Оперативная память
3. Диск
4. Сеть

🔮 **Подсказка:**

Понимание, как каждый из этих факторов влияет на работу базы данных, поможет выбрать лучший сервер для вашей нагрузки.

Ответ:

- Процессор
- Оперативная память
- Диск
- Сеть

🔮 **Шаг 12:**

Миссия 12

 **Задача:** Сопоставьте типы нагрузки на сервер с их описанием

Типы нагрузки:

Ответ:

- OLTP
- OLAP

Описания:

1. Нагрузки, ориентированные на быстрые транзакции и малые объемы данных.
2. Нагрузки, ориентированные на аналитическую обработку больших объемов данных.

 **Подсказка:**

Обратите внимание на тип задач, которые выполняются в каждой нагрузке.

- **OLTP** — Нагрузки, ориентированные на быстрые транзакции и малые объемы данных.
- **OLAP** — Нагрузки, ориентированные на аналитическую обработку больших объемов данных.

 **Шаг 13:**

Миссия 13

Что из следующего является оптимизацией для работы с большими данными в PostgreSQL?

Подсказка: Эффективная работа с большими данными требует особых подходов.

Ответ:

- **Использование партиционирования таблиц**
- Использование только индексов
- Использование только памяти
- Использование только SSD-дисков

 **Шаг 14:**

Миссия 14

Что такое партиционирование таблиц в PostgreSQL?

Подсказка: Это техника разделения больших таблиц на более мелкие части для повышения производительности.

Ответ:

- **Разделение таблицы на несколько меньших частей**
- Создание дополнительной индексации на таблице
- Использование только одного сервера для хранения всех данных
- Оптимизация запросов для работы с большими объемами данных

 **Шаг 15:**

Миссия 15

Расположите шаги для настройки репликации в PostgreSQL в правильном порядке:

1. Настройка основной базы данных
2. Настройка реплики
3. Запуск репликации

 **Подсказка:**

Чтобы репликация работала правильно, сначала необходимо настроить основную базу, а затем подключить реплику.

Ответ:

- Настройка основной базы данных
- Настройка реплики
- Запуск репликации

Шаг 16:

Миссия 16

 Задача: Сопоставьте различные виды индексов в PostgreSQL с их использованием

Виды индексов:

Ответ:

- B-tree
- GIN

Описание:

1. Используется для операций поиска в таблицах с числовыми или строковыми значениями.
2. Используется для индексирования текста и массивов.

 **Подсказка:**

Разные типы индексов применяются для разных типов данных.

- **B-tree** — Используется для операций поиска в таблицах с числовыми или строковыми значениями.
- **GIN** — Используется для индексирования текста и массивов.

Шаг 17:

Миссия 17

Какая команда PostgreSQL используется для создания индекса?

Подсказка: Для индексации вы можете использовать команду, которая запускает процессы создания в базе данных.

Ответ:

- **CREATE INDEX**
- ALTER TABLE
- SELECT INDEX
- INSERT INDEX

Шаг 18:

Миссия 18

 Задача: Сопоставьте типы блокировок в PostgreSQL с их описанием

Типы блокировок:

Ответ:

- Row-level lock
- Table-level lock

Описание:

1. Блокировка конкретной строки в таблице для предотвращения изменения данных другим процессом.
2. Блокировка всей таблицы, которая не позволяет другим процессам изменять таблицу.

💡 **Подсказка:**

Существует два основных типа блокировок: для отдельных строк и для всей таблицы.

- **Row-level lock** — Блокировка конкретной строки в таблице для предотвращения изменения данных другим процессом.
- **Table-level lock** — Блокировка всей таблицы, которая не позволяет другим процессам изменять таблицу.

📖 Урок 2 - Зачем нужен бенчмаркинг

💡 **Шаг 01:**

Миссия 1

Аня — младший разработчик, и она впервые услышала слово «бенчмаркинг». Она думает, что это про создание красивых интерфейсов. Но что на самом деле означает «бенчмаркинг» в PostgreSQL?

Подсказка: Это связано с измерением производительности, а не с дизайном кнопочек.

Ответ:

- **Сравнение производительности системы в разных условиях**
- Создание красивых интерфейсов
- Проектирование базы данных с нуля
- Добавление пользователей в систему

💡 **Шаг 02:**

Миссия 2

Игорь запускает PostgreSQL на старом ноутбуке и удивляется, почему всё тормозит. Он предполагает, что виновата СУБД. Что должен показать бенчмаркинг?

Подсказка: Иногда дело не в софте, а в железе.

Ответ:

- **Насколько ресурсы сервера соответствуют нагрузке**
- Сколько пользователей зарегистрировано в базе
- Какая версия PostgreSQL установлена
- Какие расширения подключены

💡 **Шаг 03:**

Миссия 3

Марина спорит с коллегами: нужно ли тратить время на бенчмаркинг перед запуском в прод? Какой главный аргумент в пользу бенчмаркинга?

Подсказка: Лучше заранее узнать слабые места, чем падать под нагрузкой.

Ответ:

- **Позволяет выявить узкие места до реальной нагрузки**
- Ускоряет процесс написания кода
- Заменяет мониторинг системы

- Гарантирует отсутствие ошибок в SQL

🔗 Шаг 04:

Миссия 4

Сергей хочет проверить PostgreSQL, но не знает, какую метрику измерять первой. Что является основным показателем производительности при бенчмаркинге?

Подсказка: Это связано со временем выполнения операций.

Ответ:

- **Время отклика и количество транзакций в секунду**
- Количество пользователей в базе
- Количество индексов в схеме
- Размер логов

🔗 Шаг 05:

Миссия 5

Алексей проводит тесты и замечает: в утренние часы база работает быстро, а вечером тормозит. Что объясняет такую разницу?

Подсказка: Подумайте о пиковых нагрузках и конкуренции за ресурсы.

Ответ:

- **Изменение количества параллельных пользователей**
- Изменение версии PostgreSQL
- Автоматическая очистка логов
- Смена часового пояса

🔗 Шаг 06:

Миссия 6

Дмитрий запускает `rbench` и получает результаты. Но он не уверен, что тест репрезентативен. Что важно сделать перед запуском бенчмарка?

Подсказка: Думайте о реалистичности сценария.

Ответ:

- **Смоделировать нагрузку, близкую к реальному использованию**
- Удалить все индексы из базы
- Перезапустить сервер
- Изменить код приложения

🔗 Шаг 07:

Миссия 7

Компания проводит бенчмаркинг на двух серверах: один с SSD, другой с HDD. Результаты сильно отличаются. Почему?

Подсказка: Хранение данных влияет на скорость ввода-вывода.

Ответ:

- **Разная скорость дисковой подсистемы**
- Разный цвет корпусов серверов
- Версия Linux на одном сервере новее
- Разный `timezone` в настройках

🔗 Шаг 08:

Миссия 8

Оля измеряет скорость вставки данных и получает странные результаты: при маленьком объёме всё быстро, а при миллионах записей — тормоза. Что объясняет такую ситуацию?

Подсказка: Индексы помогают, но и замедляют массовые операции.

Ответ:

- **Рост таблицы и нагрузка на индексы**
- Ошибки в SQL-синтаксисе
- Случайные баги в `pgbench`
- Смена драйвера подключения

🔗 Шаг 09:

Миссия 9

Андрей сравнивает результаты бенчмарков и замечает: иногда TPS выше, но задержка тоже больше. Что это значит?

Подсказка: Система может обрабатывать больше транзакций, но дольше отвечать конкретным пользователям.

Ответ:

- **Высокая пропускная способность с ценой в задержку**
- Ошибки в драйвере PostgreSQL
- Недостаток оперативной памяти
- Неправильные индексы

🔗 Шаг 10:

Миссия 10

📖 Задача: сопоставить метрику и её значение при бенчмаркинге.

Подсказка: TPS — это про количество, latency — про скорость отклика.

Ответ:

- **TPS (Transactions per Second)** — Количество транзакций, выполняемых в секунду
- **Latency** — Среднее время отклика запроса
- **Throughput** — Общее количество обработанных данных за единицу времени
- **Concurrency** — Количество одновременных подключений

🔗 Шаг 11:

Миссия 11

📖 Задача: Сопоставьте инструмент и его назначение.

Подсказка: Вспомните, что `pgbench` — встроенный инструмент, а `sysbench` универсальнее.

Ответ:

- **pgbench** — Стандартный инструмент бенчмаркинга PostgreSQL
- **sysbench** — Инструмент для стресс-тестирования систем и баз данных
- **iostat** — Мониторинг ввода-вывода дисков
- **htop** — Наглядное отображение загрузки процессора и памяти

Шаг 12:

Миссия 12

 Задача: Сопоставьте тип нагрузки с примером использования.

Подсказка: Чтение и запись могут быть сбалансированными или перекошенными.

Ответ:

- **OLTP** — Обработка множества мелких транзакций (например, банковские операции)
- **OLAP** — Выполнение сложных аналитических запросов
- **Read-heavy** — Сценарий, где большинство операций — это SELECT
- **Write-heavy** — Сценарий с массовыми вставками и обновлениями

Шаг 13:

Миссия 13

 Задача: Сопоставьте проблему и её возможную причину.

Подсказка: Если запросы медленные — ищите блокировки или медленные диски.

Ответ:

- **Высокая задержка** — Медленный диск или перегрузка процессора
- **Низкий TPS** — Недостаток ресурсов или конкуренция за блокировки
- **Падение производительности вечером** — Пик пользователей
- **Резкий рост использования памяти** — Отсутствие настройки autovacuum

Шаг 14:

Миссия 14

Расположите шаги бенчмаркинга в правильном порядке.

Подсказка: Сначала планирование, потом тесты, затем анализ.

Ответ:

- Определить цели бенчмаркинга
- Настроить окружение и нагрузочный сценарий
- Запустить тесты
- Проанализировать результаты

Шаг 15:

Миссия 15

Расположите действия администратора при выборе сервера для нагрузки.

Подсказка: Сначала определяются требования, потом сравниваются результаты.

Ответ:

- Определение сценария нагрузки
- Запуск тестов на разных серверах
- Сравнение показателей TPS и Latency
- Выбор оптимального сервера

Урок 3 - Виды бенчмарков

Шаг 01:

Миссия 1

Маша услышала от коллеги слово «микробенчмарк» и решила, что это что-то про маленькие базы данных. Что на самом деле означает микробенчмарк?

Подсказка: Это связано с измерением отдельных операций, а не с размером базы.

Ответ:

- **Измерение производительности отдельных операций (например, INSERT, SELECT)**
- Бенчмарк для маленьких баз
- Тестирование дизайна интерфейса
- Сравнение разных языков программирования

Шаг 02:

Миссия 2

Игорь читает статью и встречается слово «макробенчмарк». Он думает, что это связано с Excel. Что же означает макробенчмарк в PostgreSQL?

Подсказка: Это про тестирование системы целиком, а не про офисные макросы.

Ответ:

- **Тестирование производительности всей системы под нагрузкой**
- Использование макросов в SQL
- Автоматизация отчётов
- Оптимизация функций Excel

Шаг 03:

Миссия 3

Марина спорит с коллегой: что полезнее — микробенчмарк или макробенчмарк? Какое отличие между ними?

Подсказка: Один показывает детали, другой — картину целиком.

Ответ:

- **Микробенчмарк тестирует отдельные операции, а макробенчмарк — всю систему**
- Микробенчмарк быстрее, макробенчмарк медленнее
- Микробенчмарк платный, макробенчмарк бесплатный
- Микробенчмарк для PostgreSQL, макробенчмарк для MySQL

Шаг 04:

Миссия 4

Сергей запускает rpgbench и удивляется: что это за вид бенчмарка?

Подсказка: Подумайте, rpgbench моделирует транзакции, близкие к реальности.

Ответ:

- **Макробенчмарк (нагрузка, имитирующая реальных пользователей)**
- Микробенчмарк (одна операция)
- Тестирование GUI
- Функциональное тестирование

Шаг 05:

Миссия 5

Оля слышит про синтетические и реальные бенчмарки. Она запуталась. Что такое синтетический бенчмарк?

Подсказка: Здесь нет реальных пользователей, а есть искусственные сценарии.

Ответ:

- **Тестирование с искусственно созданными нагрузками**
- Тестирование с реальными пользователями
- Тестирование только SELECT-запросов
- Тестирование только INSERT-запросов

Шаг 06:

Миссия 6

Дмитрий проводит нагрузочные тесты на кластере PostgreSQL с реальными запросами приложения. Какой это вид бенчмарка?

Подсказка: Это ближе к реальности, чем искусственные сценарии.

Ответ:

- **Реалистичный (production-like) бенчмарк**
- Синтетический бенчмарк
- Функциональный тест
- Юнит-тест

Шаг 07:

Миссия 7

Алексей запускает бенчмарк, который измеряет только INSERT. Как правильно классифицировать этот тест?

Подсказка: Один тип операции = микробенчмарк.

Ответ:

- **Микробенчмарк**
- Макробенчмарк
- Синтетический бенчмарк
- Регрессионный тест

Шаг 08:

Миссия 8

Андрей читает документацию: TPC-C — это известный стандартный бенчмарк. К какому типу он относится?

Подсказка: Он имитирует реальные бизнес-процессы.

Ответ:

- **Макробенчмарк**
- Микробенчмарк
- Синтетический бенчмарк
- Юнит-тест

Шаг 09:

Миссия 9

Катя хочет проверить скорость сложных аналитических запросов (OLAP). Какой бенчмарк ей подойдёт?

Подсказка: Есть отдельные тесты для аналитики и хранилищ данных.

Ответ:

- **OLAP-бенчмарк**
- OLTP-бенчмарк
- Микробенчмарк SELECT
- Синтетический INSERT-тест

🔮 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте вид бенчмарка и его описание.

Подсказка: Микро — про детали, макро — про целое.

Ответ:

- **Микробенчмарк** — Измерение производительности отдельных операций
- **Макробенчмарк** — Оценка всей системы под нагрузкой
- **Синтетический бенчмарк** — Искусственные сценарии нагрузки
- **Реалистичный бенчмарк** — Тесты, максимально приближенные к работе приложения

🔮 Шаг 11:

Миссия 11

📖 Задача: Сопоставьте стандарт и область его применения.

Подсказка: TPC-C — про транзакции, TPC-H — про аналитику.

Ответ:

- **TPC-C** — Стандарт для транзакционных нагрузок (OLTP)
- **TPC-H** — Стандарт для аналитических запросов (OLAP)
- **pgbench** — Встроенный инструмент PostgreSQL для нагрузочного тестирования
- **sysbench** — Универсальный инструмент стресс-тестирования

🔮 Шаг 12:

Миссия 12

📖 Задача: Сопоставьте вид нагрузки и пример.

Подсказка: OLTP — это много маленьких операций, OLAP — длинные запросы.

Ответ:

- **OLTP-бенчмарк** — Множество коротких транзакций (например, интернет-магазин)
- **OLAP-бенчмарк** — Сложные запросы к хранилищу данных
- **Read-heavy тест** — Сценарий, где доминируют SELECT-запросы
- **Write-heavy тест** — Сценарий, где преобладают вставки и обновления

🔮 Шаг 13:

Миссия 13

📖 Задача: Сопоставьте инструмент и его тип бенчмарка.

Подсказка: pgbench — встроенный, TPC — стандартизированный.

Ответ:

- **pgbench** — Синтетический макробенчмарк
- **TPC-C** — Реалистичный OLTP-бенчмарк
- **TPC-H** — Реалистичный OLAP-бенчмарк

- **fio** — Бенчмарк ввода-вывода на уровне диска

Шаг 14:

Миссия 14

Расположите шаги проведения микробенчмарка в правильном порядке.

Подсказка: Сначала выбираем операцию, потом измеряем, затем анализируем.

Ответ:

- Выбор операции (например, INSERT)
- Настройка тестового окружения
- Запуск серии измерений
- Анализ времени выполнения

Шаг 15:

Миссия 15

Расположите этапы макробенчмарка.

Подсказка: Здесь больше шагов: моделирование, нагрузка, анализ.

Ответ:

- Выбор сценария нагрузки
- Запуск тестов с имитацией пользователей
- Сбор статистики (TPS, latency)
- Интерпретация результатов

Модуль 2 - Основы PostgreSQL и настройка окружения

Урок 1 - Установка PostgreSQL на разных системах

Шаг 01:

Миссия 1

Алексей решил установить PostgreSQL на Windows. Какой официальный способ установки наиболее предпочтителен?

Подсказка: Вспомните про официальный сайт PostgreSQL и удобный инсталлятор для Windows.

Ответ:

- **Использовать официальный графический установщик с сайта [postgresql.org](https://www.postgresql.org)**
- Скачать случайный архив с форума
- Собрать из исходников вручную
- Установить только через Docker, других вариантов нет

Шаг 02:

Миссия 2

Марина ставит PostgreSQL на Ubuntu. Какой пакетный менеджер ей следует использовать?

Подсказка: Вспомните стандартные инструменты Debian-подобных систем.

Ответ:

- **apt**
- yum
- dnf

- choco

🔧 Шаг 03:

Миссия 3

Иван развертывает сервер на CentOS. Какой пакетный менеджер он применит для установки PostgreSQL?

Подсказка: CentOS основан на Red Hat.

Ответ:

- **yum или dnf**
- apt
- pacman
- brew

🔧 Шаг 04:

Миссия 4

На macOS программистка Оля решила поставить PostgreSQL. Какой способ установки считается наиболее удобным?

Подсказка: Вспомните популярный пакетный менеджер для macOS.

Ответ:

- **Через Homebrew**
- Через apt
- Через yum
- Только вручную из исходников

🔧 Шаг 05:

Миссия 5

Программист Саша хочет проверить последнюю версию PostgreSQL без изменения своей основной системы. Какой способ подойдет лучше всего?

Подсказка: Вспомните про контейнеризацию.

Ответ:

- **Использовать Docker-образ PostgreSQL**
- Сразу обновить всю систему
- Удалить старую версию и поставить новую на рабочей машине
- Ждать обновления дистрибутива

🔧 Шаг 06:

Миссия 6

Сопоставьте ОС и рекомендуемый способ установки PostgreSQL.

Подсказка: Каждая ОС имеет свой пакетный менеджер или рекомендуемый инструмент.

Ответ:

- **Ubuntu** — apt
- **CentOS** — yum/dnf
- **macOS** — Homebrew
- **Windows** — Официальный графический установщик

Шаг 07:

Миссия 7

Сопоставьте инструмент и его роль при установке PostgreSQL.

Подсказка: Одни утилиты ставят пакеты, другие помогают запускать контейнеры.

Ответ:

- **apt** — Менеджер пакетов для Debian/Ubuntu
- **dnf** — Менеджер пакетов для RedHat/CentOS
- **Homebrew** — Менеджер пакетов для macOS
- **Docker** — Изолированное тестирование PostgreSQL

Шаг 08:

Миссия 8

Сопоставьте ситуацию и правильный вариант установки PostgreSQL.

Подсказка: Подумайте, кто и зачем выбирает тот или иной способ.

Ответ:

- **Нужна последняя версия для экспериментов** — Использовать Docker
- **Обычная установка на Ubuntu** — Через apt
- **Разработка на macOS** — Через Homebrew
- **Рабочая станция Windows** — Официальный установщик

Шаг 09:

Миссия 9

Сопоставьте команду и её назначение при установке PostgreSQL.

Подсказка: Вспомните типичные команды Linux-пакетных менеджеров.

Ответ:

- **apt update** — Обновление списка пакетов
- **apt install postgresql** — Установка PostgreSQL
- **yum install postgresql** — Установка PostgreSQL в CentOS
- **brew install postgresql** — Установка PostgreSQL в macOS

Шаг 10:

Миссия 10

Сопоставьте дистрибутив и характерную особенность установки PostgreSQL.

Подсказка: Обратите внимание на системные различия.

Ответ:

- **Ubuntu** — Простая установка через apt
- **CentOS** — Требуется настройка репозитория PostgreSQL
- **Windows** — Графический мастер установки
- **macOS** — Установка через brew и готовые сервисы

Шаг 11:

Миссия 11

Расположите шаги установки PostgreSQL через apt на Ubuntu в правильном порядке.

Подсказка: Вспомните: сначала обновляют список пакетов.

Ответ:

- `sudo apt update`
- `sudo apt install postgresql`
- `systemctl start postgresql`
- `psql --version`

Шаг 12:

Миссия 12

Расставьте шаги для установки PostgreSQL через yum в правильном порядке.

Подсказка: Сначала добавляют репозиторий, затем устанавливают.

Ответ:

- `sudo yum install -y [https://download.postgresql.org/...rpm`
- `sudo](https://download.postgresql.org/...rpm%22},{%22text%22:%22sudo) yum install postgresql-server`
- `sudo postgresql-setup initdb`
- `sudo systemctl start postgresql`

Шаг 13:

Миссия 13

Определите порядок установки PostgreSQL через Homebrew на macOS.

Подсказка: `brew update` всегда идёт первым.

Ответ:

- `brew update`
- `brew install postgresql`
- `brew services start postgresql`
- `psql --version`

Шаг 14:

Миссия 14

Установите порядок действий для запуска PostgreSQL в Docker.

Подсказка: Сначала тянут образ, потом запускают контейнер.

Ответ:

- `docker pull postgres`
- `docker run --name pg -e POSTGRES_PASSWORD=pass -d postgres`
- `docker ps`
- `docker exec -it pg psql -U postgres`

Шаг 15:

Миссия 15

Расположите шаги проверки установки PostgreSQL в правильном порядке.

Подсказка: Сначала проверяют службу, потом версию.

Ответ:

- `systemctl status postgresql`

- `psql --version`
- `psql -U postgres`
- Создать тестовую базу

Шаг 16:

Миссия 16

Расставьте шаги установки PostgreSQL на Windows в правильном порядке.

Подсказка: Сначала скачивают установщик, потом запускают его.

Ответ:

- Скачать установщик с [postgresql.org](https://www.postgresql.org)
- Запустить exe-файл
- Выбрать параметры установки
- Проверить `psql --version`

Шаг 17:

Миссия 17

Определите правильный порядок настройки PostgreSQL после установки.

Подсказка: Сначала создают пользователя, потом базу.

Ответ:

- `psql -U postgres`
- `CREATE USER myuser WITH PASSWORD 'pass';`
- `CREATE DATABASE mydb OWNER myuser;`
- Подключиться к `mydb`

Урок 2 - Конфигурация PostgreSQL для тестирования

Шаг 01:

Миссия 1

Аня готовит тестовый сервер для PostgreSQL. Она скачала дистрибутив, но не знает, где хранится файл конфигурации `postgresql.conf`. Какое расположение чаще всего используется по умолчанию?

Подсказка: Обычно рядом с каталогом данных базы данных.

Ответ:

- **В каталоге данных PostgreSQL (data)**
- В папке `/usr/bin`
- В папке `/etc/network`
- В домашней директории пользователя

Шаг 02:

Миссия 2

Борис запускает PostgreSQL для нагрузочного теста. Какой параметр отвечает за количество подключений, которые сервер сможет одновременно обслуживать?

Подсказка: Этот параметр ограничивает число клиентских соединений.

Ответ:

- `max_connections`

- shared_buffers
- work_mem
- checkpoint_timeout

🔗 Шаг 03:

Миссия 3

Игорь готовит сервер для тестирования. Он хочет уменьшить время записи контрольных точек. Какой параметр в postgresql.conf ему нужно изменить?

Подсказка: Этот параметр задаёт частоту сброса данных на диск.

Ответ:

- **checkpoint_timeout**
- random_page_cost
- work_mem
- wal_buffers

🔗 Шаг 04:

Миссия 4

Катя хочет проверить производительность диска при работе PostgreSQL. Какой параметр отвечает за размер буфера, используемого для записи WAL?

Подсказка: Этот параметр начинается с wal_.

Ответ:

- **wal_buffers**
- shared_buffers
- temp_buffers
- effective_cache_size

🔗 Шаг 05:

Миссия 5

Денис запускает pgbench. Какой параметр PostgreSQL влияет на использование оперативной памяти при выполнении сортировок и хешей?

Подсказка: Этот параметр задаётся на сессию и не общий для всех.

Ответ:

- **work_mem**
- shared_buffers
- maintenance_work_mem
- max_parallel_workers

🔗 Шаг 06:

Миссия 6

Мария проводит нагрузочный тест. Она хочет ускорить работу больших индексов. Какой параметр поможет выделить больше памяти именно для операций обслуживания (например, VACUUM или CREATE INDEX)?

Подсказка: Этот параметр похож на work_mem, но используется для обслуживания.

Ответ:

- **maintenance_work_mem**

- `work_mem`
- `shared_buffers`
- `temp_buffers`

🔗 Шаг 07:

Миссия 7

Сергей тестирует кэширование. Какой параметр PostgreSQL указывает планировщику запросов предполагаемый размер дискового кэша ОС?

Подсказка: Этот параметр не выделяет память, а только помогает оптимизатору.

Ответ:

- **`effective_cache_size`**
- `shared_buffers`
- `work_mem`
- `temp_buffers`

🔗 Шаг 08:

Миссия 8

Лена хочет уменьшить стоимость случайного доступа к диску в запросах. Какой параметр нужно изменить?

Подсказка: Этот параметр используется в планировщике для оценки стоимости.

Ответ:

- **`random_page_cost`**
- `seq_page_cost`
- `effective_cache_size`
- `shared_buffers`

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте параметры PostgreSQL с их назначением:

Подсказка: Внимательно подумайте: одни параметры связаны с памятью, другие с диском или соединениями.

Ответ:

- **`shared_buffers`** — Размер памяти, выделенной под кэширование данных
- **`work_mem`** — Объём памяти для сортировок и хешей в рамках сессии
- **`checkpoint_timeout`** — Частота записи контрольных точек на диск
- **`max_connections`** — Максимальное количество клиентских подключений

🔗 Шаг 10:

Миссия 10

📖 Сопоставьте параметры конфигурации с типом ресурса, на который они влияют:

Подсказка: Одни параметры касаются памяти, другие — процессоров или диска.

Ответ:

- **`effective_cache_size`** — Подсказка планировщику о размере кэша ОС
- **`random_page_cost`** — Стоимость случайного доступа к диску
- **`maintenance_work_mem`** — Объём памяти для операций обслуживания

- **max_parallel_workers** — Количество потоков для параллельных запросов

🔗 Шаг 11:

Миссия 11

📖 Сопоставьте параметр с примером его использования:

Подсказка: Вспомните реальные сценарии работы PostgreSQL.

Ответ:

- **wal_buffers** — Хранение изменений до записи в WAL-файлы
- **temp_buffers** — Временные таблицы внутри транзакций
- **seq_page_cost** — Стоимость последовательного чтения страниц
- **max_connections** — Ограничение числа одновременных клиентов

🔗 Шаг 12:

Миссия 12

📖 Сопоставьте параметры с тем, какие задачи они помогают ускорить:

Подсказка: Свяжите конкретные действия (создание индекса, сортировка) с параметрами.

Ответ:

- **work_mem** — Ускорение сортировок в запросах
- **maintenance_work_mem** — Быстрое создание индексов
- **shared_buffers** — Кэширование часто используемых данных
- **checkpoint_timeout** — Регулирование частоты записи данных

🔗 Шаг 13:

Миссия 13

📖 Сопоставьте параметр PostgreSQL с его категорией:

Подсказка: Категории — Память, Диск, Подключения, Планировщик.

Ответ:

- **max_connections** — Подключения
- **shared_buffers** — Память
- **random_page_cost** — Планировщик
- **checkpoint_timeout** — Диск

🔗 Шаг 14:

Миссия 14

Расположите шаги настройки PostgreSQL для тестирования в правильном порядке:

Подсказка: Сначала определяют цели теста, затем правят конфигурацию, перезапускают сервер и проверяют.

Ответ:

- Определить цели и метрики тестирования
- Настроить параметры postgresql.conf
- Перезапустить сервер
- Запустить нагрузочные тесты

Шаг 15:

Миссия 15

Расположите действия администратора после изменения параметров в правильной последовательности:

Подсказка: Важно помнить про сохранение, перезапуск и проверку.

Ответ:

- Изменить параметр в postgresql.conf
- Сохранить файл
- Перезапустить PostgreSQL
- Проверить результат с помощью SHOW или pg_settings

Шаг 16:

Миссия 16

Расположите шаги оптимизации памяти в PostgreSQL:

Подсказка: Сначала задают буфер, потом память для сортировок, затем для обслуживания.

Ответ:

- Установить shared_buffers
- Настроить work_mem
- Настроить maintenance_work_mem
- Проверить результат тестами

Шаг 17:

Миссия 17

Расположите шаги подготовки окружения для нагрузочного теста PostgreSQL:

Подсказка: Логика: установка, конфигурация, тест, анализ.

Ответ:

- Установить PostgreSQL
- Сконфигурировать параметры производительности
- Запустить нагрузочный инструмент (pgbench)
- Собрать и проанализировать метрики

Урок 3 - Настройка окружения для бенчмарков

Шаг 01:

Миссия 1

Андрей решил подготовить ноутбук для первых бенчмарков PostgreSQL. С чего лучше всего начать настройку окружения?

Подсказка: Начинать всегда стоит с самой базы — без неё не получится ни один тест.

Ответ:

- **Установить PostgreSQL и убедиться, что сервер запускается**
- Сразу запускать pgbench без проверки установки
- Менять параметры ядра ОС до установки
- Настраивать мониторинг без базы

🔧 Шаг 02:

Миссия 2

Марина хочет запускать бенчмарки на Linux-сервере. Какой первый шаг в подготовке окружения стоит сделать после установки PostgreSQL?

Подсказка: Прежде чем трогать конфиги, нужно проверить, что сервис вообще работает.

Ответ:

- **Проверить, что сервис PostgreSQL запущен и доступен**
- Изменить random_page_cost
- Настроить системные лимиты памяти
- Запустить VACUUM FULL

🔧 Шаг 03:

Миссия 3

Иван собрался сравнивать разные серверы для PostgreSQL. Что нужно сделать, чтобы тесты были честными и воспроизводимыми?

Подсказка: Результаты должны быть сопоставимы, иначе эксперимент теряет смысл.

Ответ:

- **Использовать одинаковые версии PostgreSQL на всех серверах**
- **Фиксировать параметры конфигурации**
- Запускать тесты в произвольное время без учета нагрузки
- Менять схемы БД между тестами

🔧 Шаг 04:

Миссия 4

Сергей планирует нагрузочное тестирование. Какой инструмент чаще всего используется для бенчмарков PostgreSQL?

Подсказка: Этот инструмент входит в стандартную поставку PostgreSQL.

Ответ:

- **pgbench**
- sysbench
- ab (Apache Benchmark)
- fio

🔧 Шаг 05:

Миссия 5

Лена решила протестировать отклик базы на 10 параллельных подключений. Какой параметр pgbench ей стоит использовать?

Подсказка: Этот параметр управляет количеством клиентов.

Ответ:

- **-c 10**
- -t 10
- -j 10
- -n 10

🔧 Шаг 06:

Миссия 6

Команда разработчиков заметила, что результаты бенчмарков сильно скачут. Что может быть причиной?

Подсказка: Иногда дело не в самой базе, а в том, что окружение живёт своей жизнью.

Ответ:

- **Фоновая нагрузка от других процессов**
- **Непоследовательные параметры конфигурации**
- Использование одной версии PostgreSQL
- Фиксированное количество подключений

🔧 Шаг 07:

Миссия 7

Сопоставьте параметры `pgbench` с их назначением:

Подсказка: Вспомните, что `-c` связано с клиентами, а `-j` с потоками.

Ответ:

- **-c** — Количество клиентов
- **-j** — Количество потоков
- **-T** — Время выполнения теста
- **-t** — Количество транзакций на клиента

🔧 Шаг 08:

Миссия 8

Сопоставьте элементы окружения с их ролью в бенчмарках:

Подсказка: Подумайте, какие части отвечают за железо, а какие за программную настройку.

Ответ:

- **CPU** — Выполнение вычислений
- **RAM** — Буферизация данных
- **Диск** — Хранение и скорость ввода-вывода
- **PostgreSQL config** — Регулировка параметров работы базы

🔧 Шаг 09:

Миссия 9

Свяжите параметры PostgreSQL с их правильным описанием:

Подсказка: `shared_buffers` управляет памятью, а `work_mem` — временными операциями.

Ответ:

- **shared_buffers** — Размер памяти, выделенной для кэширования данных
- **work_mem** — Память для сортировок и хеш-таблиц
- **max_connections** — Максимальное число одновременных подключений
- **wal_buffers** — Память для записи WAL

🔧 Шаг 10:

Миссия 10

Сопоставьте системные инструменты с их назначением при подготовке окружения:

Подсказка: `top` и `iostat` помогают смотреть за ресурсами, `pg_ctl` управляет сервером.

Ответ:

- **top** — Мониторинг нагрузки CPU и памяти
- **iostat** — Наблюдение за вводом-выводом диска
- **pg_ctl** — Управление запуском PostgreSQL
- **free** — Проверка использования оперативной памяти

Шаг 11:

Миссия 11

Сопоставьте типы бенчмарков с их описанием:

Подсказка: Один тип измеряет пиковую производительность, другой — стабильность под нагрузкой.

Ответ:

- **Синтетические тесты** — Искусственные сценарии для проверки производительности
- **Стресс-тесты** — Проверка работы под экстремальной нагрузкой
- **Длительные тесты** — Измерение стабильности на продолжительном интервале
- **Реальные сценарии** — Имитируют работу реального приложения

Шаг 12:

Миссия 12

Сопоставьте инструменты мониторинга с их описанием:

Подсказка: `pg_stat_activity` — про текущие сессии, а `vmstat` — про ресурсы ОС.

Ответ:

- **pg_stat_activity** — Список активных подключений
- **pg_stat_io** — Статистика ввода-вывода базы
- **vmstat** — Системная статистика процессов и памяти
- **htop** — Расширенный монитор ресурсов

Шаг 13:

Миссия 13

Расположите шаги подготовки окружения к бенчмарку в правильном порядке:

Подсказка: Сначала установка, потом настройка, потом проверка.

Ответ:

- Установить PostgreSQL
- Настроить параметры конфигурации
- Запустить сервис и проверить доступность
- Запустить тест `pgbench`

Шаг 14:

Миссия 14

Упорядочьте действия по настройке логирования в PostgreSQL:

Подсказка: Без включенного логирования вы не сможете анализировать результаты тестов.

Ответ:

- Открыть `postgresql.conf`

- Изменить параметры log_statement и log_duration
- Перезапустить PostgreSQL
- Запустить тест и проверить логи

Шаг 15:

Миссия 15

Расположите этапы подготовки тестовой базы в pgbench:

Подсказка: Сначала создаётся база, затем инициализируются данные.

Ответ:

- Создать тестовую базу
- Запустить pgbench -i для инициализации
- Убедиться, что таблицы созданы
- Начать нагрузочный тест

Шаг 16:

Миссия 16

Расположите шаги при анализе результатов бенчмарка:

Подсказка: Сначала собираем данные, потом их интерпретируем.

Ответ:

- Собрать результаты теста
- Посмотреть статистику PostgreSQL
- Сравнить с эталонными результатами
- Сделать вывод о производительности

Шаг 17:

Миссия 17

Расположите шаги оптимизации окружения для повторного бенчмарка:

Подсказка: После анализа обычно идут изменения и новые тесты.

Ответ:

- Проанализировать узкие места
- Изменить параметры конфигурации
- Перезапустить PostgreSQL
- Запустить повторный бенчмарк

Модуль 3 - Базовые бенчмарки PostgreSQL на разном оборудовании

Урок 1 - Бенчмарки на локальной машине

Шаг 01:

Миссия 1

Вы запустили PostgreSQL локально для первых тестов. Какой инструмент чаще всего применяют для базовых бенчмарков?

Подсказка: Этот инструмент встроен прямо в PostgreSQL и позволяет моделировать нагрузку.

Ответ:

- **pgbench**
- pg_dump
- psql
- pg_restore

🔮 **Шаг 02:**

Миссия 2

Разработчик Артём хочет сравнить производительность SSD и HDD. На какой параметр в pgbench стоит обратить внимание в первую очередь?

Подсказка: Этот параметр показывает, сколько транзакций удалось выполнить за секунду.

Ответ:

- **tps (transactions per second)**
- Размер базы
- Количество пользователей
- Версия PostgreSQL

🔮 **Шаг 03:**

Миссия 3

Маша проводит тест на своём ноутбуке. Она заметила, что результаты сильно скачут. Какое первое действие стоит предпринять?

Подсказка: Для чистых экспериментов часто приходится убирать случайные факторы.

Ответ:

- **Закрывать фоновые программы и процессы**
- Переустановить PostgreSQL
- Изменить версию pgbench
- Поменять схему базы

🔮 **Шаг 04:**

Миссия 4

Иван хочет протестировать разные уровни параллелизма. Какой параметр pgbench указывает число клиентских подключений?

Подсказка: Представьте, что вы задаёте количество виртуальных пользователей.

Ответ:

- **-c**
- -j
- -T
- -n

🔮 **Шаг 05:**

Миссия 5

Сергей заметил, что тесты слишком короткие и не отражают реальную нагрузку. Какая опция позволяет задать время работы теста?

Подсказка: Эта опция задаётся в секундах.

Ответ:

- -T
- -c
- -j
- -s

Шаг 06:

Миссия 6

Юля хочет прогреть кэш базы данных перед замером. Какой подход чаще всего используют? Подсказка: Иногда первый прогон используют только для разогрева.

Ответ:

- **Сделать несколько прогонов и использовать результаты не первого запуска**
- Увеличить размер таблиц в 10 раз
- Отключить кэширование в PostgreSQL
- Изменить алгоритм транзакций

Шаг 07:

Миссия 7

Николай проводит тесты на своём ПК и хочет, чтобы нагрузка соответствовала рабочему сценарию. Что можно сделать?

Подсказка: Иногда используют собственные SQL-скрипты вместо встроенных.

Ответ:

- **Написать кастомный сценарий для pgbench**
- Удалить все стандартные таблицы
- Запустить VACUUM FULL
- Изменить порт PostgreSQL

Шаг 08:

Миссия 8

Алексей запускает pgbench, но видит сообщение «database does not exist». Что он забыл сделать?

Подсказка: Перед тестами нужно создать специальную базу.

Ответ:

- **Инициализировать базу командой pgbench -i**
- Удалить все таблицы
- Установить расширение pg_stat_statements
- Перезапустить сервер

Шаг 09:

Миссия 9

 Задача: Сопоставьте параметры pgbench с их назначением.

Подсказка: Вспомните, что одни параметры задают количество клиентов, другие — время, а третьи — параллельные потоки.

Ответ:

- **-c** — Количество клиентских подключений
- **-T** — Длительность теста в секундах
- **-j** — Число рабочих потоков
- **-s** — Масштаб базы данных

 Шаг 10:

Миссия 10

 **Задача:** Сопоставьте команды PostgreSQL с их назначением.

Подсказка: Каждая команда отвечает за определённый этап работы с базой.

Ответ:

- **psql** — Подключение и работа с базой
- **pgbench** — Нагрузочное тестирование
- **pg_dump** — Резервное копирование базы
- **pg_restore** — Восстановление из бэкапа

 Шаг 11:

Миссия 11

 **Задача:** Сопоставьте действия с их влиянием на результаты бенчмарка.

Подсказка: Подумайте, что делает система быстрее, а что вносит шум в измерения.

Ответ:

- **Закрыть фоновые приложения** — Снижает шум в тестах
- **Добавить больше клиентов** — Повышает нагрузку на сервер
- **Использовать SSD вместо HDD** — Ускоряет доступ к данным
- **Отключить кэш ОС** — Моделирует «чистый» диск

 Шаг 12:

Миссия 12

 **Задача:** Сопоставьте метрики pgbench с их описанием.

Подсказка: Каждая метрика отвечает за определённый аспект производительности.

Ответ:

- **tps** — Количество транзакций в секунду
- **latency avg** — Среднее время выполнения запроса
- **latency stddev** — Разброс времени откликов
- **number of clients** — Количество подключённых клиентов

 Шаг 13:

Миссия 13

 **Задача:** Сопоставьте типы тестов с их целями.

Подсказка: Одни тесты проверяют пиковую нагрузку, другие — стабильность во времени.

Ответ:

- **Короткий стресс-тест** — Проверка пиковых значений
- **Длительный прогон** — Оценка стабильности
- **Тест с кастомным скриптом** — Моделирование бизнес-логики

- **Тест с увеличенным scale** — Проверка работы с большими данными

Шаг 14:

Миссия 14

Расставьте шаги запуска `pgbench` в правильном порядке:

1. Выполнить сам бенчмарк с нужными параметрами
2. Создать тестовую базу
3. Инициализировать её командой `pgbench -i`
4. Проанализировать результаты

Подсказка: Сначала нужно подготовить базу, затем выполнить тест и только потом посмотреть результаты.

Ответ:

- Создать тестовую базу
- Инициализировать её командой `pgbench -i`
- Выполнить сам бенчмарк с нужными параметрами
- Проанализировать результаты

Шаг 15:

Миссия 15

Расположите действия по подготовке ноутбука к тестам в правильном порядке:

1. Закрыть фоновые приложения
2. Проверить температуру процессора
3. Запустить PostgreSQL
4. Убедиться, что диск не перегружен

Подсказка: Подготовка окружения начинается с «чистки» системы, затем проверка состояния, потом запуск базы.

Ответ:

- Закрыть фоновые приложения
- Проверить температуру процессора
- Запустить PostgreSQL
- Убедиться, что диск не перегружен

Урок 2 - Бенчмарки на виртуальных машинах

Шаг 01:

Миссия 1

Аня разворачивает PostgreSQL на виртуальной машине и хочет провести первый тест `pgbench`. Какой параметр чаще всего используют для задания количества транзакций?

Подсказка: Этот параметр отвечает за число выполняемых операций, а не за подключений.

Ответ:

- **-t (transactions)**
- -c (clients)
- -j (jobs)
- -s (scale)

🔧 Шаг 02:

Миссия 2

Сергей запускает бенчмарк на небольшой виртуальной машине. Он заметил, что при увеличении числа клиентов нагрузка растёт, но TPS (transactions per second) падает. Что это может значить?

Подсказка: Виртуальная машина имеет ограниченные ресурсы процессора и памяти.

Ответ:

- **У виртуалки не хватает ресурсов для параллельной нагрузки**
- pgbench работает неправильно
- PostgreSQL не поддерживает многопоточность
- Ошибка в конфигурации сети

🔧 Шаг 03:

Миссия 3

Игорь хочет сравнить производительность PostgreSQL на двух разных типах виртуалок: с HDD и с SSD. Какой параметр теста даст наиболее заметную разницу?

Подсказка: Думайте о нагрузке на диск при записи.

Ответ:

- **Тесты с большим количеством записей**
- Тесты с большим количеством SELECT
- Тесты только с кэшем
- Тесты без индексов

🔧 Шаг 04:

Миссия 4

Марина использует виртуалку с ограниченным числом CPU. Она хочет параллельно прогнать pgbench с большим числом клиентов. Что произойдет?

Подсказка: Обратите внимание, что планировщик виртуализации делит ресурсы процессора.

Ответ:

- **Система начнёт сильно переключать контексты, замедляя тест**
- Производительность резко возрастет
- PostgreSQL автоматически выключится
- Результаты не изменятся

🔧 Шаг 05:

Миссия 5

Денис заметил, что на его виртуалке pgbench выдаёт сильно нестабильные результаты в разное время суток. Почему это может происходить?

Подсказка: Подумайте о том, что виртуальные машины делят общие ресурсы.

Ответ:

- **Соседние виртуальные машины создают конкуренцию за ресурсы**
- PostgreSQL случайным образом меняет алгоритмы
- pgbench некорректно считает транзакции
- Виртуальная машина изменяет лимиты сама по себе

🔮 Шаг 06:

Миссия 6

📖 Сопоставьте параметры `rgbench` с их значением. Никита готовит скрипт для теста.

Подсказка: Вспомните, что каждый параметр управляет разной частью нагрузки.

Ответ:

- **-c** — Количество клиентов
- **-j** — Количество рабочих потоков
- **-t** — Число транзакций на клиента
- **-s** — Размер базы данных в масштабах

🔮 Шаг 07:

Миссия 7

📖 Сопоставьте метрики бенчмарка с их смыслом. Лена анализирует результаты `rgbench`.

Подсказка: Каждая метрика описывает разный аспект производительности.

Ответ:

- **TPS** — Транзакции в секунду
- **latency** — Среднее время ответа на транзакцию
- **stddev latency** — Разброс времени ответа
- **initialization** — Этап подготовки тестовой базы

🔮 Шаг 08:

Миссия 8

📖 Сопоставьте тип виртуальной машины с ожидаемым эффектом для PostgreSQL. Петр выбирает конфигурацию.

Подсказка: Подумайте о том, что влияет на диск, память и CPU.

Ответ:

- **VM с SSD** — Быстрые операции записи
- **VM с HDD** — Замедленные транзакции при интенсивной записи
- **VM с большим количеством RAM** — Эффективное кэширование
- **VM с одним CPU** — Проблемы при высокой параллельности

🔮 Шаг 09:

Миссия 9

📖 Сопоставьте ситуацию с её вероятной причиной. Антон пытается понять колебания результатов.

Подсказка: Разные аспекты виртуализации влияют на стабильность.

Ответ:

- **TPS падает при росте клиентов** — Недостаточно CPU
- **Сильно растёт latency** — Медленный диск
- **Результаты колеблются** — Соседние VM загружают хост
- **Тест медленно запускается** — Малый объём RAM

🔮 Шаг 10:

Миссия 10

 Сопоставьте команды `pgbench` с их действием. Виктор готовит скрипты нагрузочного теста. Подсказка: Вспомните, какие команды отвечают за подготовку, запуск и анализ.

Ответ:

- **`pgbench -i`** — Инициализация тестовой базы
- **`pgbench -S`** — Тест только SELECT-запросов
- **`pgbench -N`** — Тест без обновления балансов
- **`pgbench -b tpcb-like`** — Запуск стандартного профиля нагрузки

Шаг 11:

Миссия 11

 Сопоставьте проблему и способ её минимизации. Катя оптимизирует тесты на виртуалке. Подсказка: Иногда помогает изменить параметры запуска, иногда — настройки VM.

Ответ:

- **Высокая задержка диска** — Использовать SSD вместо HDD
- **Мало RAM** — Увеличить память виртуалки
- **Нестабильные результаты** — Запускать тесты в одно и то же время
- **Недостаточно CPU** — Добавить виртуальные ядра

Шаг 12:

Миссия 12

 Сопоставьте вид нагрузки с тем, что она тестирует. Олег исследует разные режимы `pgbench`. Подсказка: Запросы бывают только на чтение, на запись и комбинированные.

Ответ:

- **Тест `SELECT-only`** — Максимальная скорость чтения
- **Тест `read-write`** — Баланс чтения и записи
- **Тест с большим `scale`** — Работа с большой базой
- **Тест с малым `scale`** — Работа в кэше

Шаг 13:

Миссия 13

Расположите шаги проведения бенчмарка на виртуальной машине в правильном порядке:

 **Подсказка:**

Сначала база должна быть готова, потом запускается тест, и только потом анализируются результаты.

Ответ:

- Создать и инициализировать базу
- Запустить `pgbench` с нужными параметрами
- Собрать результаты TPS и latency
- Сравнить с другими конфигурациями VM

Шаг 14:

Миссия 14

Расположите действия разработчика, который хочет протестировать `SELECT-only` нагрузку на виртуалке:

Подсказка:

Подумайте: сначала база, потом ключи запуска, и только потом анализ.

Ответ:

- Инициализировать базу данными
- Запустить `pgbench` с флагом `-S`
- Замерить TPS
- Сравнить результат с предыдущими тестами

Шаг 15:

Миссия 15

Упорядочите шаги при анализе нестабильных результатов на виртуальной машине:

Подсказка:

Сначала убедитесь в корректности теста, затем посмотрите на окружение.

Ответ:

- Проверить параметры `pgbench`
- Запустить несколько повторов
- Сравнить время запуска с активностью соседних VM
- Сделать вывод о влиянии виртуализации

Шаг 16:

Миссия 16

Расположите шаги для теста с увеличением `scale` в правильном порядке:

Подсказка:

`Scale` задаётся при инициализации, а анализ всегда в конце.

Ответ:

- Выбрать `scale`
- Инициализировать базу с этим `scale`
- Запустить нагрузочный тест
- Проанализировать рост `latency`

Урок 3 - Бенчмарки на физических серверах

Шаг 01:

Миссия 1

Андрей установил PostgreSQL на физический сервер и хочет проверить базовую производительность. Какой первый шаг он должен сделать?

Подсказка: Начинать стоит с проверки простых операций ввода-вывода.

Ответ:

- **Запустить `pgbench` с простым сценарием `SELECT 1`**
- Сразу запускать нагрузку с миллионами транзакций
- Настроить кластер репликации
- Сравнить разные версии PostgreSQL на другом сервере

Шаг 02:

Миссия 2

Мария запускает `pgbench` на сервере с большим количеством ядер. Что важно проверить при старте?

Подсказка: Помните про количество клиентских соединений.

Ответ:

- **Соответствие числа клиентов количеству ядер**
- Чтобы сервер работал только на одном ядре
- Чтобы `pgbench` не выполнял ни одной транзакции
- Чтобы лог-файл был пустым

Шаг 03:

Миссия 3

Иван заметил, что производительность падает при использовании HDD. Что станет очевидным при бенчмарках?

Подсказка: Вспомните про задержки записи и случайные операции.

Ответ:

- **Медленные случайные чтения и записи**
- Высокая пропускная способность как у SSD
- Скорость растёт при добавлении нагрузки
- Производительность не зависит от диска

Шаг 04:

Миссия 4

Сергей тестирует сервер с большим количеством оперативной памяти. Что может показать бенчмарк?

Подсказка: Вспомните, когда данные помещаются в кеш.

Ответ:

- **Снижение числа обращений к диску при достаточном кеше**
- Увеличение задержки на запись
- Обязательное падение производительности
- Бенчмарк всегда игнорирует кеш

Шаг 05:

Миссия 5

Ольга хочет сравнить производительность на серверах с разными сетевыми картами. Как правильно организовать тест?

Подсказка: Сеть тоже влияет на результат.

Ответ:

- **Проводить `pgbench` с удалёнными клиентами**
- Измерять только загрузку CPU
- Игнорировать сетевые задержки
- Запускать бенчмарк только на `localhost`

Шаг 06:

Миссия 6

Сопоставьте компоненты сервера с типичными эффектами на результат бенчмарка PostgreSQL.

Подсказка: Вспомните, какие ресурсы чаще всего становятся узким местом.

Ответ:

- **CPU с большим количеством ядер** — Выше производительность при параллельной нагрузке
- **SSD** — Быстрые случайные операции
- **HDD** — Высокие задержки на случайных операциях
- **Большой объём RAM** — Часть базы помещается в кеш

🔮 Шаг 07:

Миссия 7

Сопоставьте параметры `pgbench` с их назначением.

Подсказка: Каждый ключ управляет разными аспектами нагрузки.

Ответ:

- **-c** — Количество клиентов
- **-j** — Количество потоков
- **-T** — Время теста
- **-f** — Использовать пользовательский сценарий

🔮 Шаг 08:

Миссия 8

Программисты проводят сравнение серверов. Сопоставьте тип нагрузки с наиболее важным ресурсом.

Подсказка: Разные тесты нагружают разные подсистемы.

Ответ:

- **OLTP нагрузка** — CPU и транзакционная система
- **Большие аналитические запросы** — Пропускная способность диска
- **Мелкие UPDATE в цикле** — Скорость записи
- **Множество параллельных SELECT** — Количество соединений и ядра

🔮 Шаг 09:

Миссия 9

Сопоставьте метрики бенчмарка с их смыслом.

Подсказка: Вспомните, какие показатели выдают `pgbench` и `monitoring`.

Ответ:

- **tps** — Количество транзакций в секунду
- **latency** — Среднее время отклика
- **95th percentile** — 95% запросов быстрее этого значения
- **throughput** — Пропускная способность

🔮 Шаг 10:

Миссия 10

Сопоставьте тип сервера с его особенностями при бенчмарках.

Подсказка: Помните про различия между поколениями оборудования.

Ответ:

- **Сервер с DDR4 RAM** — Более высокая скорость работы памяти
- **Сервер со старыми HDD** — Долгие задержки при случайном доступе
- **Сервер с NVMe** — Очень быстрый доступ к данным
- **Сервер с низкой тактовой частотой CPU** — Проблемы в OLTP нагрузках

🔗 Шаг 11:

Миссия 11

Сопоставьте задачи администраторов с корректными действиями при бенчмарках.
Подсказка: Вспомните практические шаги при тестировании.

Ответ:

- **Измерить влияние кеша ОС** — Запускать тесты на горячей и холодной базе
- **Понять роль сетевой карты** — Тестировать с внешнего клиента
- **Выявить лимит по CPU** — Увеличить число клиентов
- **Проверить запись на диск** — Использовать сценарии с INSERT/UPDATE

🔗 Шаг 12:

Миссия 12

Сопоставьте распространённые ошибки при бенчмарках с их последствиями.
Подсказка: Ошибки в методике часто искажают результат.

Ответ:

- **Слишком мало клиентов** — Недогруз процессора
- **Тест только на localhost** — Игнорирование сетевых задержек
- **Короткое время теста** — Результаты нестабильны
- **Отсутствие повторов** — Нет статистической достоверности

🔗 Шаг 13:

Миссия 13

Расположите шаги подготовки сервера к бенчмарку в правильном порядке.
Подсказка: Сначала установка и настройка, потом подготовка данных и запуск.

Ответ:

- Установить PostgreSQL
- Настроить параметры shared_buffers и work_mem
- Инициализировать тестовую базу
- Запустить pgbench

🔗 Шаг 14:

Миссия 14

Расположите шаги анализа результатов бенчмарка.
Подсказка: Сначала собираем данные, затем сравниваем и делаем выводы.

Ответ:

- Собрать tps и latency
- Сравнить с другими конфигурациями
- Выявить узкие места

- Сделать выводы и рекомендации

Шаг 15:

Миссия 15

Расположите шаги сравнения двух физических серверов.

Подсказка: Тесты должны выполняться в одинаковых условиях.

Ответ:

- Запустить одинаковые тесты на обоих серверах
- Собрать метрики tps и latency
- Построить таблицу сравнения
- Сделать вывод о лучшем сервере

Шаг 16:

Миссия 16

Расположите шаги выявления влияния дисковой подсистемы.

Подсказка: Сначала нагружаем диск, потом оцениваем задержки.

Ответ:

- Создать сценарий с активными INSERT
- Запустить тесты на HDD
- Запустить тесты на SSD
- Сравнить задержки и tps

Шаг 17:

Миссия 17

Расположите шаги итогового выбора сервера для PostgreSQL.

Подсказка: Сначала тесты, потом анализ и решение.

Ответ:

- Провести серию бенчмарков
- Сравнить результаты на разных серверах
- Определить узкие места
- Выбрать оптимальный сервер

Модуль 4 - Оптимизация запросов и серверов

Урок 1 - Оптимизация простых запросов

Шаг 01:

Миссия 1

Алексей заметил, что его `SELECT * FROM orders` работает медленно. Что он должен сделать в первую очередь, чтобы ускорить запрос?

Подсказка: Начинать всегда лучше с анализа плана выполнения, а не с угадываний.

Ответ:

- **Посмотреть EXPLAIN и изучить план выполнения**
- Сразу добавить индекс на все столбцы
- Увеличить количество оперативной памяти сервера

- Удалить все старые данные из таблицы

🔮 Шаг 02:

Миссия 2

Марина оптимизирует запрос с фильтром WHERE status = 'active'. Что поможет PostgreSQL ускорить этот запрос?

Подсказка: Сравни, что будет эффективнее — полный просмотр таблицы или узкая выборка.

Ответ:

- **Создание индекса по полю status**
- Перенос таблицы в другую схему
- Добавление лишних JOIN-ов
- Увеличение размера транзакции

🔮 Шаг 03:

Миссия 3

Разработчик Игорь пишет запрос с ORDER BY created_at DESC LIMIT 10. Что сделает его быстрее?

Подсказка: Вспомни про индексы и их порядок сортировки.

Ответ:

- **Использование индекса по полю created_at DESC**
- Добавление индекса на все столбцы таблицы
- Переписывание LIMIT на OFFSET
- Запуск ANALYZE только один раз в год

🔮 Шаг 04:

Миссия 4

Ольга запускает сложный JOIN запрос и он работает медленно. Какой первый шаг стоит предпринять?

Подсказка: Иногда не код менять нужно, а статистику обновить.

Ответ:

- **Выполнить ANALYZE для актуализации статистики**
- Увеличить количество колонок в SELECT
- Поменять все JOIN на подзапросы
- Запустить VACUUM FULL каждые 5 минут

🔮 Шаг 05:

Миссия 5

Команда тестирует запрос SELECT COUNT(*) FROM users. Почему он может быть медленным на большой таблице?

Подсказка: Вспомни, что COUNT без условий требует просмотра всех строк.

Ответ:

- **Потому что PostgreSQL считает каждую строку**
- Потому что PostgreSQL всегда использует кэш
- Потому что таблица не имеет первичного ключа
- Потому что запрос выполняется только на мастере

🔮 Шаг 06:

Миссия 6

Сопоставьте инструмент оптимизации PostgreSQL с его назначением.

Подсказка: Один отвечает за статистику, другой — за физическую очистку.

Ответ:

- **ANALYZE** — Обновляет статистику для планировщика
- **VACUUM** — Очищает пространство от мертвых строк
- **EXPLAIN** — Показывает план выполнения запроса
- **INDEX** — Структура для ускорения поиска данных

🔮 Шаг 07:

Миссия 7

Программист Дима изучает планы выполнения. Соотнесите термин с его смыслом.

Подсказка: Вспомни, что Seq Scan ≠ Index Scan.

Ответ:

- **Seq Scan** — Последовательное чтение всей таблицы
- **Index Scan** — Поиск строк по индексу
- **Nested Loop** — Выполнение вложенных циклов JOIN
- **Hash Join** — Сопоставление через хэш-таблицу

🔮 Шаг 08:

Миссия 8

Сопоставьте тип индекса с его особенностью.

Подсказка: Один хорош для равенств, другой для полнотекстового поиска.

Ответ:

- **B-Tree** — Подходит для поиска по диапазонам и равенствам
- **GIN** — Эффективен для полнотекстового поиска
- **BRIN** — Хорош для очень больших таблиц с упорядоченными данными
- **Hash Index** — Оптимален для поиска по равенствам

🔮 Шаг 09:

Миссия 9

Инженер Катя анализирует работу кэша. Соотнесите понятие с определением.

Подсказка: shared_buffers — это сердце PostgreSQL-кэша.

Ответ:

- **shared_buffers** — Память, выделенная под кэш данных
- **work_mem** — Используется для сортировок и хеш-таблиц
- **maintenance_work_mem** — Память для VACUUM, CREATE INDEX
- **effective_cache_size** — Оценка доступного дискового кэша ОС

🔮 Шаг 10:

Миссия 10

Сопоставьте вид JOIN с его результатом.

Подсказка: Подумай, где строки исчезают, а где добавляются NULL-ы.

Ответ:

- **INNER JOIN** — Возвращает только совпадающие строки
- **LEFT JOIN** — Возвращает все строки из левой таблицы и совпадения из правой
- **RIGHT JOIN** — Возвращает все строки из правой таблицы и совпадения из левой
- **FULL JOIN** — Возвращает все строки из обеих таблиц

Шаг 11:

Миссия 11

Компания тестирует индексацию. Соотнесите команду с её действием.

Подсказка: CREATE INDEX ≠ REINDEX.

Ответ:

- **CREATE INDEX** — Создание нового индекса
- **DROP INDEX** — Удаление индекса
- **REINDEX** — Пересоздание поврежденного или устаревшего индекса
- **ALTER INDEX** — Изменение параметров существующего индекса

Шаг 12:

Миссия 12

Соотнесите тип агрегатной функции с её примером.

Подсказка: SUM и AVG легко перепутать.

Ответ:

- **SUM** — Суммирует значения
- **AVG** — Вычисляет среднее
- **MIN** — Находит минимальное значение
- **MAX** — Находит максимальное значение

Шаг 13:

Миссия 13

Расположите шаги оптимизации запроса в правильном порядке.

Подсказка: Сначала анализ, потом изменения, потом проверка.

Ответ:

- Посмотреть EXPLAIN
- Добавить нужные индексы
- Запустить ANALYZE
- Проверить время выполнения после оптимизации

Шаг 14:

Миссия 14

Расположите действия по настройке work_mem.

Подсказка: Настройки сначала анализируют, потом корректируют.

Ответ:

- Изучить план выполнения запроса
- Определить, хватает ли памяти для сортировки

- Изменить значение work_mem
- Повторно протестировать запрос

Шаг 15:

Миссия 15

Упорядочите шаги создания частичного индекса.

Подсказка: Частичный индекс создается только на часть строк.

Ответ:

- Определить условие WHERE
- Составить команду CREATE INDEX
- Выполнить индексирование
- Проверить ускорение запросов

Шаг 16:

Миссия 16

Расположите шаги оптимизации COUNT(*).

Подсказка: Иногда вместо подсчета всех строк используют специальные подходы.

Ответ:

- Определить, зачем нужен точный COUNT
- Рассмотреть возможность использования метаданных
- Применить агрегацию с условием
- Сравнить результаты и время выполнения

Шаг 17:

Миссия 17

Упорядочите действия по настройке PostgreSQL для долгосрочной оптимизации.

Подсказка: Сначала замеряем, потом меняем параметры, потом наблюдаем.

Ответ:

- Замерить производительность текущих запросов
- Изменить параметры конфигурации
- Выполнить нагрузочное тестирование
- Наблюдать за долгосрочными метриками

Урок 2 - Оптимизация сложных запросов

Шаг 01:

Миссия 1

Программист Анна запускает сложный SQL-запрос с несколькими JOIN, и он работает медленно. Что первым делом стоит проверить?

Подсказка: Начни с самой простой диагностики: не всегда проблема в самом запросе.

Ответ:

- **План выполнения запроса (EXPLAIN)**
- Сразу переписать запрос с подзапросами
- Запустить VACUUM FULL без анализа
- Сменить сервер на более мощный без анализа

Шаг 02:

Миссия 2

Разработчик Игорь заметил, что подзапрос в WHERE сильно замедляет работу. Что можно сделать?

Подсказка: Иногда переписывание запроса на JOIN вместо подзапроса даёт выигрыш.

Ответ:

- **Переписать подзапрос в JOIN**
- Добавить лишние индексы на все поля
- Удалить условие WHERE совсем
- Запустить запрос в транзакции без коммита

Шаг 03:

Миссия 3

Сергей анализирует запрос с ORDER BY и LIMIT 10, который выполняется слишком долго. Что может ускорить выполнение?

Подсказка: Иногда сортировка может стать лёгкой задачей при правильной структуре индекса.

Ответ:

- **Создать индекс, учитывающий порядок сортировки**
- Добавить OFFSET 1000
- Запускать запрос ночью
- Убрать LIMIT

Шаг 04:

Миссия 4

Мария хочет ускорить запрос с множественными агрегатами (SUM, COUNT). Какой приём может помочь?

Подсказка: Иногда проще подготовить данные заранее, чем считать их каждый раз.

Ответ:

- **Использовать материализованное представление**
- Дублировать таблицу много раз
- Выполнять несколько одинаковых запросов подряд
- Заменить агрегаты на подстроки

Шаг 05:

Миссия 5

Александр работает с запросом, где много условий AND и OR. Запрос стал слишком сложным. Что стоит сделать?

Подсказка: Иногда разбиение задачи на части помогает оптимизатору.

Ответ:

- **Разбить запрос на несколько более простых**
- Объединить все условия в одно длинное выражение
- Использовать только OR
- Скопировать таблицу во временную

🔮 Шаг 06:

Миссия 6

📖 Задача: Сопоставьте методы оптимизации с их описанием.

Подсказка: Вспомни, что индексы помогают искать быстрее, а кэш экономит ресурсы.

Ответ:

- **Индексирование** — Ускоряет поиск по ключевым полям
- **Материализованное представление** — Хранит предварительно рассчитанные данные
- **Кэширование запросов** — Использует сохранённые результаты повторных запросов
- **Разбиение таблицы (partitioning)** — Разделяет данные для ускорения выборки

🔮 Шаг 07:

Миссия 7

📖 Задача: Сопоставьте проблему с возможным решением.

Подсказка: Иногда ответ скрыт в том, как хранятся данные.

Ответ:

- **Медленная сортировка** — Создание индекса по сортируемому столбцу
- **Долгие JOIN** — Проверка правильности индексов по ключам
- **Большие агрегаты** — Использование материализованных представлений
- **Сложные условия WHERE** — Разделение запроса на несколько этапов

🔮 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте тип индекса с его назначением.

Подсказка: Не все индексы одинаково полезны, у каждого своя роль.

Ответ:

- **B-Tree индекс** — Эффективен для равенства и диапазонов
- **GIN индекс** — Полезен для полнотекстового поиска
- **BRIN индекс** — Подходит для очень больших таблиц с упорядоченными данными
- **Hash индекс** — Оптимален для простых сравнений равенства

🔮 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте инструмент с его назначением в PostgreSQL.

Подсказка: Эти команды — твои друзья при диагностике.

Ответ:

- **EXPLAIN** — Показывает план выполнения запроса
- **ANALYZE** — Собирает статистику о таблицах
- **VACUUM** — Очищает неиспользуемое пространство
- **pg_stat_activity** — Отображает активные запросы

🔮 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте ситуацию с правильной стратегией.

Подсказка: Смотри на характер нагрузки: чтение или запись?

Ответ:

- **Частые SELECT без обновлений** — Можно активно кэшировать
- **Частые INSERT** — Полезно рассмотреть партиционирование
- **Большие отчёты** — Подойдут материализованные представления
- **Сложные JOIN** — Нужна проверка индексов

Шаг 11:

Миссия 11

 Задача: Сопоставьте разработчика с его ошибкой при работе с запросами.

Подсказка: Ошибки часто повторяются у разных людей, главное — видеть закономерности.

Ответ:

- **Аня** — Не использует EXPLAIN для анализа
- **Игорь** — Добавил индекс на каждое поле без нужды
- **Мария** — Считает агрегаты напрямую при каждом запросе
- **Сергей** — Не учитывает порядок сортировки при индексации

Шаг 12:

Миссия 12

Расположите шаги оптимизации сложного запроса в правильном порядке:

 **Подсказка:**

Сначала анализируем, потом переписываем, и только после проверяем на практике.

Ответ:

- Анализ плана выполнения
- Переписывание запроса (упрощение подзапросов)
- Создание индексов
- Повторная проверка с EXPLAIN ANALYZE

Шаг 13:

Миссия 13

Упорядочите действия администратора при обнаружении долгого запроса:

 **Подсказка:**

Не спеши — сначала диагностика, потом оптимизация.

Ответ:

- Запуск EXPLAIN
- Анализ индексов
- Переписывание запроса
- Тестирование нового решения

Шаг 14:

Миссия 14

Расположите шаги по созданию материализованного представления:

 **Подсказка:**

Сначала нужно определить, что хранить, потом создать, а после поддерживать в актуальном состоянии.

Ответ:

- Определение часто используемого запроса
- Создание материализованного представления
- Регулярное обновление REFRESH
- Использование в реальных запросах

 **Шаг 15:**

Миссия 15

Расположите шаги проверки индексации для запроса:

 **Подсказка:**

Нужно сначала понять структуру запроса, потом проверить план и только затем индексировать.

Ответ:

- Анализ WHERE и ORDER BY
- Проверка плана с EXPLAIN
- Создание нужного индекса
- Тестирование скорости выполнения

 **Шаг 16:**

Миссия 16

Упорядочите этапы диагностики сложного JOIN-запроса:

 **Подсказка:**

Сначала нужно понять, что связывается, потом — как, и лишь после внести изменения.

Ответ:

- Определение таблиц и ключей
- Анализ плана выполнения
- Проверка наличия индексов
- Оптимизация структуры JOIN

 **Шаг 17:**

Миссия 17

Расположите шаги в процессе настройки PostgreSQL под нагрузку сложных запросов:

 **Подсказка:**

Не стоит сразу менять конфиги — сначала изучаем поведение запросов.

Ответ:

- Сбор статистики запросов
- Оптимизация SQL-запросов
- Настройка параметров сервера (work_mem, shared_buffers)
- Нагрузочное тестирование

Урок 3 - Рекомендации по выбору железа для сложных запросов

 **Шаг 01:**

Миссия 1

Сергей разворачивает PostgreSQL на новом сервере и хочет выбрать оптимальный диск для OLAP-нагрузки. Что важнее всего?

Подсказка: Диск должен быстро справляться с большими объемами чтения.

Ответ:

- **SSD NVMe**
- HDD 5400 rpm
- USB флешка
- CD-ROM

Шаг 02:

Миссия 2

Алина запускает PostgreSQL на сервере с 16 ядрами и 32 ГБ RAM. Она замечает, что при сложных запросах CPU полностью загружен, а RAM почти свободна. Что ей стоит увеличить в первую очередь?

Подсказка: Обратите внимание на узкое место, где нагрузка максимальна.

Ответ:

- **Количество CPU**
- Объем RAM
- Скорость сети
- Размер диска

Шаг 03:

Миссия 3

Игорь выбирает сервер для PostgreSQL с интенсивной аналитической нагрузкой. Что он должен проверить в первую очередь?

Подсказка: Подумайте о частых операциях: сканирование таблиц и агрегаты.

Ответ:

- **Пропускную способность дисков и IO**
- Наличие RAID 0
- Дизайн корпуса сервера
- Цвет корпуса

Шаг 04:

Миссия 4

 Сопоставьте характеристики серверного железа с их влиянием на PostgreSQL.

Подсказка: Обратите внимание, как CPU, RAM и диск влияют на разные типы нагрузки.

Ответ:

- **SSD диск** — Ускоряет операции чтения и записи больших таблиц
- **Большой объем RAM** — Позволяет кэшировать данные и уменьшить обращения к диску
- **Многоядерный CPU** — Обработывает сложные параллельные запросы быстрее
- **Сеть 1Gbps** — Влияет на скорость репликации и клиентские подключения

Шаг 05:

Миссия 5

 Сопоставьте настройки PostgreSQL с их эффектом на производительность.

Подсказка: Каждая настройка влияет на разные уровни нагрузки: память, диски, параллелизм.

Ответ:

- **work_mem** — Объем памяти для сортировки и хеширования в одном запросе
- **shared_buffers** — Размер кэша PostgreSQL для данных и индексов
- **max_parallel_workers** — Количество процессов для выполнения параллельных запросов
- **effective_cache_size** — Оценка ОС для кэширования данных в памяти

Шаг 06:

Миссия 6

 Сопоставьте типы нагрузки с рекомендуемым железом.

Подсказка: OLTP и OLAP имеют разные приоритеты: диски против CPU/RAM.

Ответ:

- **OLTP** — Нужны быстрые транзакции, много RAM и CPU
- **OLAP** — Нужны SSD и много CPU для аналитики
- **Резервное копирование** — Важен быстрый диск и сеть
- **Репликация** — Важна сеть и стабильность CPU

Шаг 07:

Миссия 7

Расположите шаги оптимизации сервера PostgreSQL для аналитических запросов в правильном порядке:

1. Выбор SSD для быстрого доступа к таблицам
2. Настройка параметров work_mem и shared_buffers
3. Мониторинг CPU и I/O после нагрузки
4. Добавление параллельных воркеров для сложных запросов

Подсказка: Сначала готовим железо, потом настраиваем память и параллелизм, а в конце — проверяем и корректируем.

Ответ:

- Выбор SSD для быстрого доступа к таблицам
- Настройка параметров work_mem и shared_buffers
- Мониторинг CPU и I/O после нагрузки
- Добавление параллельных воркеров для сложных запросов

Шаг 08:

Миссия 8

 Сопоставьте типы индексов с их использованием в PostgreSQL.

Подсказка: Индексы ускоряют разные виды запросов.

Ответ:

- **B-tree** — Для быстрых точных совпадений и диапазонных запросов
- **Hash** — Для быстрых точных совпадений
- **GIN** — Для полнотекстового поиска и массивов
- **BRIN** — Для больших таблиц с упорядоченными данными

🔧 Шаг 09:

Миссия 9

📖 Сопоставьте метрики мониторинга PostgreSQL с тем, что они показывают.

Подсказка: Внимательно посмотрите, какая метрика за что отвечает.

Ответ:

- **CPU usage** — Нагрузка на процессор от запросов
- **IO wait** — Время ожидания дисковых операций
- **Buffer hit ratio** — Процент попаданий в кэш памяти
- **Active connections** — Количество одновременно подключённых клиентов

🔧 Шаг 10:

Миссия 10

📖 Сопоставьте виды сложных запросов с проблемой, которая чаще всего возникает.

Подсказка: Подумайте, какие операции чаще всего тормозят сервер.

Ответ:

- **JOIN на больших таблицах** — Высокая нагрузка на CPU и память
- **Агрегации GROUP BY** — Большой расход RAM и временных файлов на диске
- **Сортировки ORDER BY** — Нагрузка на work_mem и возможное обращение к диску
- **Подзапросы** — Сложные планы выполнения и много I/O

🔧 Шаг 11:

Миссия 11

📖 Сопоставьте сценарий нагрузки с оптимальной настройкой PostgreSQL.

Подсказка: Настройка должна соответствовать типу нагрузки.

Ответ:

- **Много коротких транзакций** — Увеличить max_connections и уменьшить work_mem
- **Сложные аналитические запросы** — Увеличить work_mem и parallel_workers
- **Резервное копирование** — Настроить checkpoint_segments и wal_buffers
- **Репликация** — Настроить wal_level и max_wal_senders

🔧 Шаг 12:

Миссия 12

📖 Сопоставьте типы хранилища с их преимуществами для PostgreSQL.

Подсказка: Разные диски подходят для разных задач.

Ответ:

- **HDD** — Дешевое, медленное, подходит для архивных данных
- **SATA SSD** — Быстрее HDD, подходит для среднего уровня нагрузки
- **NVMe SSD** — Очень быстрый доступ для OLAP и heavy I/O
- **RAID 10** — Комбинация скорости и надежности

🔧 Шаг 13:

Миссия 13

Расположите этапы подготовки сервера PostgreSQL для OLAP в правильном порядке:

1. Выбор SSD NVMe для дисковой подсистемы
2. Настройка `shared_buffers` и `work_mem`
3. Тестирование производительности и корректировка настроек
4. Добавление параллельных воркеров и настройка `max_parallel_workers`

Подсказка: Сначала готовим железо, потом память, потом параллелизм и проверку.

Ответ:

- Выбор SSD NVMe для дисковой подсистемы
- Настройка `shared_buffers` и `work_mem`
- Тестирование производительности и корректировка настроек
- Добавление параллельных воркеров и настройка `max_parallel_workers`

Шаг 14:

Миссия 14

Расположите шаги анализа медленного запроса в правильном порядке:

1. Сбор плана выполнения EXPLAIN
2. Выявление узкого места: CPU, I/O или память
3. Применение индексов или изменения запроса
4. Повторная проверка производительности

Подсказка: Сначала смотрим план, потом ищем проблему, исправляем и проверяем.

Ответ:

- Сбор плана выполнения EXPLAIN
- Выявление узкого места: CPU, I/O или память
- Применение индексов или изменения запроса
- Повторная проверка производительности

Шаг 15:

Миссия 15

Расположите шаги по подготовке резервного сервера PostgreSQL:

1. Настройка репликации и `wal_level`
2. Тестирование синхронизации с основным сервером
3. Настройка мониторинга и алертов
4. Периодическое тестирование резервного сервера

Подсказка: Репликация и проверка работоспособности — ключевые шаги.

Ответ:

- Настройка репликации и `wal_level`
- Тестирование синхронизации с основным сервером
- Настройка мониторинга и алертов
- Периодическое тестирование резервного сервера

Шаг 16:

Миссия 16

Расположите шаги по оптимизации OLTP-нагрузки:

1. Увеличение RAM для кэширования данных
2. Настройка max_connections и connection pooling
3. Индексация колонок, участвующих в WHERE и JOIN
4. Мониторинг производительности и корректировка настроек

Подсказка: Начинаем с памяти, потом пул соединений, индексы и проверка.

Ответ:

- Увеличение RAM для кэширования данных
- Настройка max_connections и connection pooling
- Индексация колонок, участвующих в WHERE и JOIN
- Мониторинг производительности и корректировка настроек

 **Шаг 17:**

Миссия 17

Расположите шаги при подготовке сервера PostgreSQL для смешанной нагрузки OLTP+OLAP:

1. Выбор сбалансированного CPU и RAM
2. Настройка shared_buffers и work_mem
3. Оптимизация индексов для часто используемых запросов
4. Мониторинг всех ресурсов и корректировка конфигурации

Подсказка: Баланс между CPU, RAM и диском — основа смешанной нагрузки.

Ответ:

- Выбор сбалансированного CPU и RAM
- Настройка shared_buffers и work_mem
- Оптимизация индексов для часто используемых запросов
- Мониторинг всех ресурсов и корректировка конфигурации

Модуль 5 - Масштабируемость и производительность

Урок 1 - Распределение нагрузки и кластеризация

 **Шаг 01:**

Миссия 1

Анна разворачивает кластер PostgreSQL для интернет-магазина с высокой нагрузкой. Что поможет равномерно распределять нагрузку между узлами?

Подсказка: Подумайте о способе направления запросов на разные серверы.

Ответ:

- **Балансировщик нагрузки**
- Увеличение CPU на одном узле
- Добавление RAM без шардирования
- Использование только SSD на одном сервере

 **Шаг 02:**

Миссия 2

Максим настраивает репликацию PostgreSQL. Что критично для уменьшения задержки при чтении с реплик?

Подсказка: Подумайте о синхронности данных между основным сервером и репликами.

Ответ:

- **Синхронная репликация**
- Отложенная репликация на сутки
- Размещение всех данных на одном сервере
- Использование медленного HDD

Шаг 03:

Миссия 3

Игорь хочет, чтобы кластер PostgreSQL легко масштабировался. Что важнее всего на этапе проектирования?

Подсказка: Подумайте о возможности добавлять новые узлы без остановки системы.

Ответ:

- **Шардирование и архитектура master-slave или multi-master**
- Установка SSD на каждом узле
- Увеличение количества индексов
- Настройка VACUUM по расписанию

Шаг 04:

Миссия 4

 Сопоставьте типы репликации PostgreSQL с их характеристиками.

Подсказка: Каждый вид репликации имеет свои плюсы и минусы.

Ответ:

- **Синхронная репликация** — Гарантирует, что данные на реплике идентичны основному серверу
- **Асинхронная репликация** — Запись на реплику может отставать, но минимальная задержка на основном сервере
- **Логическая репликация** — Позволяет реплицировать только выбранные таблицы или данные
- **Физическая репликация** — Полная копия всего кластера, включая все таблицы и настройки

Шаг 05:

Миссия 5

 Сопоставьте типы нагрузки с рекомендациями по распределению в кластере.

Подсказка: Подумайте, какие запросы лучше направлять на реплики, а какие — на основной сервер.

Ответ:

- **OLTP-запросы** — Направлять на основной сервер для консистентности
- **OLAP-запросы** — Можно направлять на реплики для снижения нагрузки на мастер
- **Чтение часто обновляемых данных** — Лучше на основной сервер, чтобы видеть актуальные данные
- **Архивные запросы** — Можно на отдельные реплики или архивные ноды

🔮 Шаг 06:

Миссия 6

📖 Сопоставьте элементы кластера с их функцией.

Подсказка: Каждый компонент играет свою роль в распределении нагрузки.

Ответ:

- **Load Balancer** — Распределяет запросы между репликами
- **Primary Node** — Обработывает запись и критически важные транзакции
- **Replica Node** — Обработывает чтение и уменьшает нагрузку на основной сервер
- **Backup Node** — Хранит резервные копии для аварийного восстановления

🔮 Шаг 07:

Миссия 7

📖 Сопоставьте типы шардирования с подходящими сценариями.

Подсказка: Разные схемы шардирования подходят для разных паттернов данных.

Ответ:

- **По диапазону ключей (range)** — Подходит для исторических данных с временными метками
- **По хэшу (hash)** — Обеспечивает равномерное распределение нагрузки между узлами
- **По списку значений (list)** — Используется для категориальных данных, например, регионы
- **Гибридное шардирование** — Комбинация методов для сложных схем распределения

🔮 Шаг 08:

Миссия 8

📖 Сопоставьте метрики мониторинга кластера с их значением.

Подсказка: Каждая метрика отражает конкретную проблему или узкое место.

Ответ:

- **Replication lag** — Задержка между основным сервером и репликой
- **CPU usage** — Нагрузка на процессор всех узлов
- **Disk I/O** — Производительность операций чтения/записи
- **Active connections** — Количество текущих подключений к кластеру

🔮 Шаг 09:

Миссия 9

📖 Сопоставьте типы запросов с оптимальным распределением между узлами.

Подсказка: Некоторые запросы можно параллелить, другие требуют консистентности.

Ответ:

- **Транзакции обновления** — На основной узел для консистентности
- **Сложные аналитические запросы** — На реплики для снижения нагрузки на мастер
- **Отчеты и агрегации** — На отдельные аналитические узлы
- **Чтение архивных данных** — На реплики или холодные ноды

🔮 Шаг 10:

Миссия 10

Расположите шаги по развертыванию кластера PostgreSQL с балансировкой нагрузки в правильном порядке:

1. Выбор архитектуры master-slave или multi-master
2. Настройка балансировщика нагрузки
3. Настройка репликации между узлами
4. Тестирование распределения запросов и мониторинг

Подсказка: Сначала проектируем архитектуру, затем настраиваем компоненты, потом проверяем.

Ответ:

- Выбор архитектуры master-slave или multi-master
- Настройка балансировщика нагрузки
- Настройка репликации между узлами
- Тестирование распределения запросов и мониторинг

Шаг 11:

Миссия 11

Расположите шаги по настройке шардирования для нового кластера:

1. Определение ключа шардирования
2. Выбор метода шардирования (range, hash, list)
3. Разделение данных между узлами
4. Тестирование распределения и мониторинг нагрузки

Подсказка: Сначала выбираем стратегию, потом распределяем данные и проверяем балансировку.

Ответ:

- Определение ключа шардирования
- Выбор метода шардирования (range, hash, list)
- Разделение данных между узлами
- Тестирование распределения и мониторинг нагрузки

Шаг 12:

Миссия 12

Расположите шаги по мониторингу кластера PostgreSQL:

1. Сбор метрик CPU, RAM и Disk I/O
2. Сбор метрик репликации и задержек
3. Анализ узких мест и определение проблемных узлов
4. Корректировка конфигурации и балансировки нагрузки

Подсказка: Мониторинг — непрерывный процесс: сначала сбор данных, потом анализ и исправление.

Ответ:

- Сбор метрик CPU, RAM и Disk I/O
- Сбор метрик репликации и задержек
- Анализ узких мест и определение проблемных узлов
- Корректировка конфигурации и балансировки нагрузки

Шаг 13:

Миссия 13

Расположите шаги по добавлению нового узла в кластер:

1. Подготовка нового сервера и установка PostgreSQL
2. Подключение к балансировщику нагрузки
3. Настройка репликации с основным узлом
4. Тестирование корректного распределения запросов

Подсказка: Новый узел добавляется постепенно: подготовка, подключение, репликация, тестирование.

Ответ:

- Подготовка нового сервера и установка PostgreSQL
- Подключение к балансировщику нагрузки
- Настройка репликации с основным узлом
- Тестирование корректного распределения запросов

Шаг 14:

Миссия 14

Расположите шаги по аварийному восстановлению кластера:

1. Определение проблемного узла
2. Переключение на резервный узел или реплику
3. Восстановление данных на новом узле
4. Возврат к нормальной работе и проверка целостности

Подсказка: Аварийное восстановление — сначала переключаем нагрузку, потом восстанавливаем и проверяем.

Ответ:

- Определение проблемного узла
- Переключение на резервный узел или реплику
- Восстановление данных на новом узле
- Возврат к нормальной работе и проверка целостности

Шаг 15:

Миссия 15

Расположите шаги по оптимизации производительности кластера:

1. Индексация колонок для частых запросов
2. Настройка параметров `shared_buffers` и `work_mem`
3. Распределение запросов через балансировщик нагрузки
4. Мониторинг и корректировка конфигурации при росте нагрузки

Подсказка: Оптимизация — комплексный процесс: сначала индексы и память, потом распределение и мониторинг.

Ответ:

- Индексация колонок для частых запросов
- Настройка параметров `shared_buffers` и `work_mem`
- Распределение запросов через балансировщик нагрузки

- Мониторинг и корректировка конфигурации при росте нагрузки

Шаг 16:

Миссия 16

Расположите шаги по плановому обновлению кластера PostgreSQL:

1. Создание резервной копии данных
2. Тестовое обновление на отдельной ноде
3. Пошаговое обновление всех узлов
4. Проверка целостности данных и корректности работы системы

Подсказка: Обновление требует сначала резервной копии, потом тестов, постепенного внедрения и проверки.

Ответ:

- Создание резервной копии данных
- Тестовое обновление на отдельной ноде
- Пошаговое обновление всех узлов
- Проверка целостности данных и корректности работы системы

Шаг 17:

Миссия 17

Расположите шаги по масштабированию кластера при росте нагрузки:

1. Анализ текущей нагрузки и выявление узких мест
2. Добавление новых реплик для чтения
3. Настройка балансировщика нагрузки для новых узлов
4. Мониторинг и корректировка конфигурации по мере роста нагрузки

Подсказка: Масштабирование — это постепенный процесс: сначала анализ, потом добавление ресурсов и мониторинг.

Ответ:

- Анализ текущей нагрузки и выявление узких мест
- Добавление новых реплик для чтения
- Настройка балансировщика нагрузки для новых узлов
- Мониторинг и корректировка конфигурации по мере роста нагрузки

Урок 2 - Масштабируемость на горизонтальном уровне

Шаг 01:

Миссия 1

Саша разворачивает горизонтально масштабируемый кластер PostgreSQL для веб-приложения с ростом пользователей. Что поможет распределить нагрузку между узлами?

Подсказка: Подумайте о направлении запросов на разные серверы.

Ответ:

- **Балансировщик нагрузки**
- Добавление RAM на одном узле
- Установка SSD без репликации
- Ограничение количества подключений

Шаг 02:

Миссия 2

Ирина хочет, чтобы новые пользователи не перегружали основной сервер. Какой подход позволит горизонтально масштабировать чтение?

Подсказка: Подумайте о репликах и распределении чтения.

Ответ:

- **Чтение с реплик**
- Увеличение CPU на основном сервере
- Добавление индексов только на мастер-узле
- Использование одного SSD для всех узлов

Шаг 03:

Миссия 3

Петр разрабатывает кластер с большим количеством транзакций. Что важно для горизонтальной масштабируемости записи?

Подсказка: Подумайте о шардировании и распределении данных.

Ответ:

- **Шардирование данных по ключу**
- Добавление памяти на одном узле
- Использование одного мастера для всех запросов
- Только создание резервных копий

Шаг 04:

Миссия 4

 Сопоставьте типы шардирования с их характеристиками.

Подсказка: Разные схемы шардирования подходят для разных паттернов данных.

Ответ:

- **Range Sharding** — Данные делятся по диапазону значений, например, по дате
- **Hash Sharding** — Равномерно распределяет данные между узлами с помощью хэш-функции
- **List Sharding** — Деление по списку категорий, например, по региону
- **Composite Sharding** — Комбинирует несколько методов для сложной структуры данных

Шаг 05:

Миссия 5

 Сопоставьте элементы горизонтального кластера PostgreSQL с их функцией.

Подсказка: Каждый компонент играет свою роль в масштабировании.

Ответ:

- **Primary Node** — Обрабатывает все операции записи
- **Replica Node** — Обрабатывает чтение и уменьшает нагрузку на мастер
- **Load Balancer** — Распределяет запросы между узлами кластера
- **Backup Node** — Хранит резервные копии для восстановления

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте типы нагрузки с оптимальным распределением между узлами.

Подсказка: OLTP и OLAP нагрузки требуют разного подхода.

Ответ:

- **OLTP-запросы** — Лучше на основной узел для консистентности данных
- **OLAP-запросы** — Можно на реплики или аналитические узлы
- **Чтение часто обновляемых данных** — На основной узел, чтобы видеть актуальные данные
- **Архивные запросы** — На отдельные узлы или холодные реплики

🔧 Шаг 07:

Миссия 7

📖 Сопоставьте метрики мониторинга кластера с их значением.

Подсказка: Метрики помогут выявить узкие места.

Ответ:

- **Replication lag** — Задержка между мастером и репликой
- **CPU usage** — Нагрузка на процессор всех узлов
- **Disk I/O** — Производительность операций чтения/записи
- **Active connections** — Количество текущих подключений к кластеру

🔧 Шаг 08:

Миссия 8

📖 Сопоставьте подходы масштабирования с их характеристиками.

Подсказка: Горизонтальное масштабирование отличается от вертикального.

Ответ:

- **Горизонтальное масштабирование** — Добавление новых узлов для увеличения производительности
- **Вертикальное масштабирование** — Увеличение ресурсов одного узла (CPU, RAM)
- **Шардирование** — Разделение данных по ключу между узлами
- **Репликация** — Создание копий для чтения и резервирования

🔧 Шаг 09:

Миссия 9

Расположите шаги по развертыванию горизонтального кластера PostgreSQL в правильном порядке:

1. Выбор архитектуры и стратегии шардирования
2. Установка PostgreSQL на новые узлы
3. Настройка балансировщика нагрузки
4. Настройка репликации и шардирования
5. Тестирование корректного распределения нагрузки

Подсказка: Сначала проектируем архитектуру, потом разворачиваем узлы и проверяем распределение.

Ответ:

- Выбор архитектуры и стратегии шардирования
- Установка PostgreSQL на новые узлы
- Настройка балансировщика нагрузки
- Настройка репликации и шардирования
- Тестирование корректного распределения нагрузки

Шаг 10:

Миссия 10

Анна хочет минимизировать простой приложения при сбое узла кластера. Какой подход горизонтального масштабирования поможет?

Подсказка: Думайте о высокой доступности и отказоустойчивости.

Ответ:

- **Добавление реплик с автоматическим failover**
- Увеличение CPU на одном узле
- Использование одного мастера без реплик
- Только резервное копирование на диске

Шаг 11:

Миссия 11

Михаил разрабатывает масштабируемый кластер. Что уменьшит задержку при частых запросах к репликам?

Подсказка: Подумайте о географическом расположении узлов и кэше.

Ответ:

- **Географическое распределение реплик и локальный кэш**
- Все узлы в одном датацентре
- Удаление индексов
- Использование одной большой машины

Шаг 12:

Миссия 12

Сергей хочет снизить нагрузку на основной узел. Что подойдет для распределения чтения?

Подсказка: Думайте о репликах и разделении ролей.

Ответ:

- **Назначение реплик только для чтения**
- Увеличение памяти на основном узле
- Добавление SSD на мастере
- Сжатие таблиц

Шаг 13:

Миссия 13

 Сопоставьте типы репликации с их характеристиками.

Подсказка: Разные типы репликации подходят для разных сценариев.

Ответ:

- **Синхронная репликация** — Мастер ждет подтверждения записи от реплики
- **Асинхронная репликация** — Мастер не ждет подтверждения, данные могут быть немного отсталыми
- **Логическая репликация** — Передает только определенные таблицы или данные
- **Физическая репликация** — Копирует весь бинарный журнал базы

Шаг 14:

Миссия 14

 Сопоставьте методы балансировки нагрузки с их назначением.

Подсказка: Балансировка влияет на чтение и запись.

Ответ:

- **Round-robin** — Равномерное распределение запросов между узлами
- **Least connections** — Направляет запрос на узел с наименьшей текущей нагрузкой
- **IP-hash** — Запросы от одного клиента всегда на один узел
- **Random** — Случайное распределение запросов

Шаг 15:

Миссия 15

 Сопоставьте подходы к шардированию с их преимуществами.

Подсказка: Каждая схема шардирования подходит под определенные данные.

Ответ:

- **Range Sharding** — Простое разделение по значениям, удобно для временных данных
- **Hash Sharding** — Равномерная нагрузка на все узлы
- **List Sharding** — Разделение по дискретным категориям, например, регион
- **Composite Sharding** — Гибкость за счет комбинирования методов

Шаг 16:

Миссия 16

Расположите шаги настройки асинхронной репликации PostgreSQL в правильном порядке:

1. Настройка wal_level и параметров репликации
2. Создание резервной копии мастера и инициализация реплики
3. Запуск реплики и проверка синхронизации
4. Мониторинг репликации и производительности

Подсказка: Сначала конфигурируем мастер, потом создаем реплику, запускаем и мониторим.

Ответ:

- Настройка wal_level и параметров репликации
- Создание резервной копии мастера и инициализация реплики
- Запуск реплики и проверка синхронизации
- Мониторинг репликации и производительности

Шаг 17:

Миссия 17

Расположите шаги развертывания шардированного кластера PostgreSQL:

1. Анализ объема данных и нагрузки
2. Выбор стратегии шардирования
3. Развертывание новых узлов и настройка шардов
4. Настройка балансировщика нагрузки и репликации
5. Тестирование корректного распределения данных и запросов

Подсказка: Сначала проектируем, потом разворачиваем узлы и проверяем.

Ответ:

- Анализ объема данных и нагрузки
- Выбор стратегии шардирования
- Развертывание новых узлов и настройка шардов
- Настройка балансировщика нагрузки и репликации
- Тестирование корректного распределения данных и запросов

Урок 3 - Тестирование нагрузки

Шаг 01:

Миссия 1

Алексей проводит нагрузочное тестирование PostgreSQL. Что лучше всего позволяет смоделировать рост количества пользователей?

Подсказка: Подумайте о генерации одновременных соединений.

Ответ:

- **Использование нагрузочного тестера, создающего параллельные запросы**
- Увеличение RAM на сервере
- Ручной запуск одного запроса за раз
- Установка индексов без нагрузки

Шаг 02:

Миссия 2

Ирина хочет проверить, как сервер PostgreSQL реагирует на пиковые нагрузки. Какой метод тестирования подходит?

Подсказка: Подумайте о временных всплесках запросов.

Ответ:

- **Spike-тестирование с кратковременными пиковыми запросами**
- Увеличение памяти сервера
- Тестирование с одним пользователем
- Добавление индексов без нагрузки

Шаг 03:

Миссия 3

Сергей планирует проверить устойчивость базы к длительным нагрузкам. Какой вид теста подойдет?

Подсказка: Думайте о долгосрочной стабильности под нагрузкой.

Ответ:

- **Endurance-тест (тест на продолжительное время)**

- Тестирование с одним запросом
- Мгновенное увеличение RAM
- Spike-тестирование на 5 секунд

🔮 Шаг 04:

Миссия 4

📖 Сопоставьте тип тестирования нагрузки с его характеристикой.

Подсказка: Каждая методика проверяет разные аспекты производительности.

Ответ:

- **Load Testing** — Проверка производительности при ожидаемой нормальной нагрузке
- **Stress Testing** — Проверка системы при нагрузке выше ожидаемой, до отказа
- **Spike Testing** — Внезапные резкие пики нагрузки
- **Endurance Testing** — Долговременное тестирование стабильности под нагрузкой

🔮 Шаг 05:

Миссия 5

📖 Сопоставьте метрику PostgreSQL с тем, что она показывает при нагрузке.

Подсказка: Мониторинг помогает выявить узкие места.

Ответ:

- **CPU usage** — Загрузка процессора при выполнении запросов
- **Disk I/O** — Производительность операций чтения/записи на диске
- **Replication lag** — Задержка между мастером и репликой
- **Active connections** — Количество одновременных подключений

🔮 Шаг 06:

Миссия 6

📖 Сопоставьте инструмент тестирования нагрузки с его особенностью.

Подсказка: Каждый инструмент имеет свои сценарии применения.

Ответ:

- **pgbench** — Инструмент для стандартного тестирования PostgreSQL
- **JMeter** — Универсальный генератор нагрузки для веб-приложений
- **Locust** — Python-инструмент для распределенного нагрузочного тестирования
- **Gatling** — Высокопроизводительный тестер для HTTP и DB

🔮 Шаг 07:

Миссия 7

📖 Сопоставьте сценарий тестирования с его целью.

Подсказка: Подумайте, какие проблемы выявляются при разных нагрузках.

Ответ:

- **Высокий трафик на короткий промежуток** — Выявление пиковых нагрузок (spike testing)
- **Длительное выполнение операций** — Проверка стабильности (endurance testing)
- **Нагрузка в пределах нормы** — Определение производительности под стандартной нагрузкой (load testing)
- **Ненормально высокая нагрузка** — Проверка отказоустойчивости (stress testing)

Шаг 08:

Миссия 8

Расположите шаги нагрузочного тестирования PostgreSQL в правильном порядке:

1. Определение сценариев нагрузки
2. Настройка тестового окружения
3. Запуск теста и генерация нагрузки
4. Сбор и анализ метрик
5. Корректировка параметров сервера и повторное тестирование

Подсказка: Сначала планируем, потом тестируем, затем анализируем и улучшаем.

Ответ:

- Определение сценариев нагрузки
- Настройка тестового окружения
- Запуск теста и генерация нагрузки
- Сбор и анализ метрик
- Корректировка параметров сервера и повторное тестирование

Шаг 09:

Миссия 9

Наталья хочет определить, как быстро PostgreSQL обрабатывает сложные JOIN-запросы. Что следует сделать сначала?

Подсказка: Сначала важно понять нагрузку, а потом измерять время выполнения.

Ответ:

- **Создать тестовую базу с типовой нагрузкой и измерить время выполнения**
- Увеличить количество индексов
- Перезапустить сервер PostgreSQL
- Сразу запускать многопоточную нагрузку

Шаг 10:

Миссия 10

Алексей хочет протестировать базу на количество одновременных подключений. Что лучше всего использовать?

Подсказка: Думайте о нагрузочном инструменте, который поддерживает множество соединений.

Ответ:

- **pgbench с параметром -c для количества клиентов**
- Скрипт с одним подключением
- Мониторинг CPU без нагрузки
- Установка новых индексов

Шаг 11:

Миссия 11

 Сопоставьте тип нагрузки с её целью тестирования PostgreSQL.

Подсказка: Разные типы нагрузки выявляют разные узкие места.

Ответ:

- **Одновременные соединения** — Проверка масштабируемости по числу клиентов
- **Долгие запросы** — Проверка устойчивости к длительной нагрузке
- **Кратковременные пики** — Выявление пиковых проблем с производительностью
- **Стандартная нагрузка** — Оценка производительности при нормальной работе

🔧 Шаг 12:

Миссия 12

📖 Сопоставьте метрику PostgreSQL с её значением при стресс-тестировании.

Подсказка: Обратите внимание, что разные метрики указывают на разные узкие места.

Ответ:

- **Buffer hit ratio** — Доля запросов, обслуживаемых из памяти
- **Deadlocks** — Количество взаимных блокировок транзакций
- **Temp files created** — Количество временных файлов для сортировки и агрегаций
- **Locks held** — Количество удерживаемых блокировок на таблицах/строках

🔧 Шаг 13:

Миссия 13

📖 Сопоставьте инструмент с его подходом к нагрузочному тестированию.

Подсказка: Каждый инструмент специализируется на разных сценариях нагрузки.

Ответ:

- **pgbench** — Генерация SQL-запросов на PostgreSQL
- **HammerDB** — Симуляция OLTP и OLAP нагрузок
- **Locust** — Создание распределённой нагрузки через Python-сценарии
- **Gatling** — Высокопроизводительный тест веб-сервисов

🔧 Шаг 14:

Миссия 14

Расположите действия по анализу результатов нагрузочного теста PostgreSQL:

1. Сбор логов и метрик
2. Визуализация графиков загрузки CPU, I/O, памяти
3. Выявление узких мест и ошибок
4. Корректировка конфигурации и повторное тестирование

Подсказка: Сначала собираем данные, потом анализируем и исправляем.

Ответ:

- Сбор логов и метрик
- Визуализация графиков загрузки CPU, I/O, памяти
- Выявление узких мест и ошибок
- Корректировка конфигурации и повторное тестирование

🔧 Шаг 15:

Миссия 15

Расположите шаги подготовки PostgreSQL к длительному тестированию нагрузки:

1. Настройка параметров памяти и WAL
2. Создание тестовых таблиц и данных
3. Определение сценариев запросов
4. Запуск теста и мониторинг системы
5. Анализ метрик и корректировка настроек

Подсказка: Сначала готовим сервер и данные, потом тестируем и анализируем.

Ответ:

- Настройка параметров памяти и WAL
- Создание тестовых таблиц и данных
- Определение сценариев запросов
- Запуск теста и мониторинг системы
- Анализ метрик и корректировка настроек

Шаг 16:

Миссия 16

Расположите действия при spike-тестировании PostgreSQL:

1. Определение пиковых сценариев нагрузки
2. Настройка инструментов генерации нагрузки
3. Запуск теста с резкими пиками
4. Сбор логов и мониторинг метрик
5. Анализ результатов и выявление узких мест

Подсказка: Spike-тест выявляет кратковременные пиковые нагрузки и их влияние на систему.

Ответ:

- Определение пиковых сценариев нагрузки
- Настройка инструментов генерации нагрузки
- Запуск теста с резкими пиками
- Сбор логов и мониторинг метрик
- Анализ результатов и выявление узких мест

Шаг 17:

Миссия 17

Расположите шаги по подготовке сервера к stress-тесту:

1. Резервное копирование данных
2. Настройка параметров максимальной нагрузки
3. Создание тестовой среды
4. Запуск стресс-теста
5. Анализ сбоев и ограничений

Подсказка: Сначала создаём защиту и тестовую среду, потом тестируем и анализируем сбои.

Ответ:

- Резервное копирование данных
- Настройка параметров максимальной нагрузки
- Создание тестовой среды

- Запуск стресс-теста
- Анализ сбоев и ограничений

Модуль 6 - Практическая часть Выбор сервера под реальные задачи

Урок 1 - Бенчмарки на основе реальных данных

Шаг 01:

Миссия 1

Иван тестирует новый сервер для PostgreSQL. Он хочет понять, какой из серверов быстрее справится с простыми SELECT-запросами. Что ему стоит сделать первым?

Подсказка: Начните с малого: сначала проверьте базовую производительность без сложных сценариев.

Ответ:

- **Запустить несколько простых SELECT-запросов и замерить время**
- Сразу тестировать сложные JOIN-операции с миллионами строк
- Менять настройки PostgreSQL без тестирования
- Сразу покупать самый дорогой сервер

Шаг 02:

Миссия 2

Мария хочет выбрать сервер для OLTP-нагрузки. Какой параметр сервера ей важнее всего в первую очередь?

Подсказка: Для транзакций важна скорость отклика и обработка большого числа операций в секунду.

Ответ:

- **IOPS и скорость работы с диском**
- Возможность запуска тяжелых аналитических запросов
- Встроенная графическая карта
- Число USB-портов

Шаг 03:

Миссия 3

Петр проверяет производительность сервера при нагрузке 1000 одновременных пользователей. Какой инструмент ему лучше использовать?

Подсказка: Существуют специальные бенчмарки для PostgreSQL, которые моделируют многопользовательские сценарии.

Ответ:

- **pgbench**
- Photoshop
- Excel
- Chrome DevTools

Шаг 04:

Миссия 4

 **Задача:** Сопоставьте тип нагрузки с оптимальным параметром сервера PostgreSQL.

Понятия:

Ответ:

- OLTP
- OLAP
- Смешанная нагрузка
- Резервное копирование

Определения:

1. Высокая скорость транзакций и небольшие записи
2. Массивные аналитические запросы на большие объемы данных
3. Баланс между транзакциями и аналитикой
4. Минимизировать нагрузку на основную систему и обеспечить быстрый бэкап

 **Подсказка:**

Подумайте о том, какая нагрузка требует быстрого отклика, а какая — обработки больших объемов.

- **OLTP** — Высокая скорость транзакций и небольшие записи
- **OLAP** — Массивные аналитические запросы на большие объемы данных
- **Смешанная нагрузка** — Баланс между транзакциями и аналитикой
- **Резервное копирование** — Минимизировать нагрузку на основную систему и обеспечить быстрый бэкап

 **Шаг 05:**

Миссия 5

 **Задача:** Сопоставьте тип индекса PostgreSQL с его особенностью.

Понятия:

Ответ:

- B-tree
- Hash
- GIN
- BRIN

Определения:

1. Подходит для точного поиска и равномерных значений
2. Хорош для полнотекстового поиска и массивов
3. Эффективен для диапазонных запросов на больших таблицах
4. Используется для сортировки и диапазонных условий

 **Подсказка:**

Подумайте, какие индексы лучше подходят для поиска по тексту, а какие — для числовых диапазонов.

- **B-tree** — Используется для сортировки и диапазонных условий
- **Hash** — Подходит для точного поиска и равномерных значений
- **GIN** — Хорош для полнотекстового поиска и массивов
- **BRIN** — Эффективен для диапазонных запросов на больших таблицах

🔧 Шаг 06:

Миссия 6

📖 Задача: Сопоставьте метрику производительности с её назначением.

Понятия:

Ответ:

- TPS (transactions per second)
- Latency
- Throughput
- IOPS

Определения:

1. Количество транзакций в секунду
2. Задержка отклика сервера
3. Объем данных, обработанных за единицу времени
4. Количество операций ввода-вывода в секунду

🔧 Подсказка:

Подумайте, какая метрика отражает скорость отклика, а какая — способность обрабатывать большие объемы операций.

- **TPS (transactions per second)** — Количество транзакций в секунду
- **Latency** — Задержка отклика сервера
- **Throughput** — Объем данных, обработанных за единицу времени
- **IOPS** — Количество операций ввода-вывода в секунду

🔧 Шаг 07:

Миссия 7

Расположите шаги подготовки сервера PostgreSQL под OLTP-нагрузку в правильном порядке:

1. Определение характеристик нагрузки (количество транзакций, размер данных)
2. Настройка параметров памяти, кешей и параллельных соединений
3. Запуск тестов и измерение TPS и latency
4. Анализ результатов и корректировка настроек

🔧 Подсказка:

Сначала анализируем, потом настраиваем, затем тестируем и корректируем.

Ответ:

- Определение характеристик нагрузки (количество транзакций, размер данных)
- Настройка параметров памяти, кешей и параллельных соединений
- Запуск тестов и измерение TPS и latency
- Анализ результатов и корректировка настроек

🔧 Шаг 08:

Миссия 8

Алексей хочет протестировать скорость вставки миллионов строк в PostgreSQL. Что ему важно настроить в первую очередь?

Подсказка: Подумайте, что влияет на скорость массовых вставок в базе данных.

Ответ:

- **Параметры wal_level и commit**
- Цвет интерфейса pgAdmin
- Размер экрана монитора
- Количество USB-портов на сервере

 **Шаг 09:**

Миссия 9

Наталья оценивает сервер для аналитических запросов (OLAP). Какой параметр сервера имеет первостепенное значение?

Подсказка: OLAP-запросы часто работают с большими таблицами, нужно больше памяти и дисковой пропускной способности.

Ответ:

- **Объем RAM и пропускная способность диска**
- Наличие встроенного Wi-Fi
- Число USB-портов
- Цвет корпуса сервера

 **Шаг 10:**

Миссия 10

Игорь хочет понять, какой сервер справляется лучше с многопоточными запросами. Какой подход наиболее корректен?

Подсказка: Используйте реальные сценарии с несколькими параллельными соединениями.

Ответ:

- **Запуск pgbench с множеством клиентов одновременно**
- Тестирование одного запроса на сервере
- Измерение температуры процессора
- Запуск скрипта на Python без базы

 **Шаг 11:**

Миссия 11

 **Задача:** Сопоставьте сценарий нагрузки с подходящей метрикой.

Понятия:

Ответ:

- Транзакции в интернет-магазине
- Анализ больших данных
- Резервное копирование ночью
- Одновременные соединения API

Определения:

1. TPS (transactions per second)
2. Throughput (объем обработанных данных)
3. IOPS и минимальная нагрузка на базу
4. Latency (задержка отклика сервера)

 **Подсказка:**

Подумайте, что важно для скорости транзакций, больших объемов данных и параллельных соединений.

- **Транзакции в интернет-магазине** — TPS (transactions per second)
- **Анализ больших данных** — Throughput (объем обработанных данных)
- **Резервное копирование ночью** — IOPS и минимальная нагрузка на базу
- **Одновременные соединения API** — Latency (задержка отклика сервера)

🔮 Шаг 12:

Миссия 12

📖 Задача: Сопоставьте тип таблицы PostgreSQL с её особенностью.

Понятия:

Ответ:

- Heap
- Unlogged
- Partitioned
- Temporary

Определения:

1. Основной тип таблицы с обычным хранением данных
2. Таблица без записи WAL, быстрее для вставок
3. Деление большой таблицы на части для оптимизации запросов
4. Временная таблица на сессию, автоматически очищается

🔮 Подсказка:

Подумайте, какая таблица предназначена для больших данных, а какая — для временных задач.

- **Heap** — Основной тип таблицы с обычным хранением данных
- **Unlogged** — Таблица без записи WAL, быстрее для вставок
- **Partitioned** — Деление большой таблицы на части для оптимизации запросов
- **Temporary** — Временная таблица на сессию, автоматически очищается

🔮 Шаг 13:

Миссия 13

📖 Задача: Сопоставьте настройку PostgreSQL с её эффектом на производительность.

Понятия:

Ответ:

- shared_buffers
- work_mem
- maintenance_work_mem
- max_connections

Определения:

1. Память для кэширования часто используемых данных
2. Память на сортировку и агрегацию отдельных запросов
3. Память для операций VACUUM и CREATE INDEX
4. Максимальное количество одновременных соединений

🔮 Подсказка:

Подумайте, какие параметры важны для быстрого выполнения запросов и операций обслуживания базы.

- **shared_buffers** — Память для кэширования часто используемых данных
- **work_mem** — Память на сортировку и агрегацию отдельных запросов
- **maintenance_work_mem** — Память для операций VACUUM и CREATE INDEX
- **max_connections** — Максимальное количество одновременных соединений

🔮 Шаг 14:

Миссия 14

Расположите шаги оптимизации PostgreSQL для аналитических запросов (OLAP) в правильном порядке:

1. Выбор подходящей схемы данных и индексов
2. Настройка памяти и параллельных воркеров
3. Запуск тестовых запросов на больших объемах данных
4. Анализ результатов и корректировка настроек

🔮 Подсказка:

Сначала проектируем структуру, потом настраиваем сервер, тестируем и корректируем.

Ответ:

- Выбор подходящей схемы данных и индексов
- Настройка памяти и параллельных воркеров
- Запуск тестовых запросов на больших объемах данных
- Анализ результатов и корректировка настроек

🔮 Шаг 15:

Миссия 15

Расположите шаги подготовки теста на многопользовательскую нагрузку:

1. Определение числа параллельных клиентов и операций
2. Создание тестовой базы и таблиц
3. Запуск `pgbench` с выбранными параметрами
4. Сбор и анализ статистики

🔮 Подсказка:

Сначала готовим данные и настройки, затем запускаем тест и анализируем результаты.

Ответ:

- Определение числа параллельных клиентов и операций
- Создание тестовой базы и таблиц
- Запуск `pgbench` с выбранными параметрами
- Сбор и анализ статистики

🔮 Шаг 16:

Миссия 16

Расположите шаги анализа результатов бенчмарка PostgreSQL:

1. Сбор логов и статистики
2. Построение графиков TPS и latency
3. Сравнение разных конфигураций сервера

4. Вывод рекомендаций по оптимизации

Подсказка:

Сначала собираем данные, потом визуализируем, сравниваем и делаем выводы.

Ответ:

- Сбор логов и статистики
- Построение графиков TPS и latency
- Сравнение разных конфигураций сервера
- Вывод рекомендаций по оптимизации

Шаг 17:

Миссия 17

Расположите шаги при выборе сервера для смешанной нагрузки (OLTP+OLAP):

1. Определение ключевых сценариев нагрузки
2. Сравнение производительности серверов на каждом сценарии
3. Настройка параметров памяти, процессоров и дисков
4. Повторное тестирование и анализ компромиссов

Подсказка:

Сначала выясняем сценарии, затем тестируем, настраиваем и проверяем баланс между OLTP и OLAP.

Ответ:

- Определение ключевых сценариев нагрузки
- Сравнение производительности серверов на каждом сценарии
- Настройка параметров памяти, процессоров и дисков
- Повторное тестирование и анализ компромиссов

Урок 2 - Создание отчета по выбору сервера

Шаг 01:

Миссия 1

Андрей подготовил результаты тестов серверов и хочет выбрать ключевые показатели для отчета. С чего лучше начать?

Подсказка: Сначала выберите метрики, которые лучше всего отражают производительность.

Ответ:

- **Выбрать основные метрики (TPS, latency, IOPS)**
- Сразу писать рекомендации без данных
- Сравнить только цвет корпуса серверов
- Добавлять в отчет все параметры BIOS

Шаг 02:

Миссия 2

Ольга хочет включить в отчет графики по производительности серверов. Какой тип графика лучше выбрать для TPS?

Подсказка: Подумайте, как визуально показать различия между серверами.

Ответ:

- **Столбчатая диаграмма**
- Круговая диаграмма
- Диаграмма рассеяния без осей
- Таблица со всеми настройками BIOS

 **Шаг 03:**

Миссия 3

Сергей собрал данные по OLTP и OLAP нагрузкам. Какую структуру отчета лучше выбрать?
Подсказка: Логично разделить результаты по типам нагрузки для наглядности.

Ответ:

- **Разделить отчет на секции: OLTP и OLAP**
- Смешать все результаты в один список
- Писать отчет только словами, без данных
- Добавлять случайные тесты серверов

 **Шаг 04:**

Миссия 4

 **Задача:** Сопоставьте метрику с её назначением в отчете по серверу.

Понятия:

Ответ:

- TPS
- Latency
- Throughput
- IOPS

Определения:

1. Количество транзакций в секунду
2. Время отклика сервера
3. Объем обработанных данных за единицу времени
4. Количество операций ввода-вывода в секунду

 **Подсказка:**

Подумайте, какая метрика показывает скорость операций, а какая — объем обработанных данных.

- **TPS** — Количество транзакций в секунду
- **Latency** — Время отклика сервера
- **Throughput** — Объем обработанных данных за единицу времени
- **IOPS** — Количество операций ввода-вывода в секунду

 **Шаг 05:**

Миссия 5

 **Задача:** Сопоставьте раздел отчета с его содержанием.

Понятия:

Ответ:

- Введение
- Методология тестирования
- Результаты и графики
- Выводы и рекомендации

Определения:

1. Описание цели отчета и проверяемых серверов
2. Подробности проведения бенчмарков и нагрузок
3. Таблицы, графики, сравнения серверов
4. Итоговые рекомендации по выбору сервера

🔗 Подсказка:

Подумайте, что логично включить в начале, середине и конце отчета.

- **Введение** — Описание цели отчета и проверяемых серверов
- **Методология тестирования** — Подробности проведения бенчмарков и нагрузок
- **Результаты и графики** — Таблицы, графики, сравнения серверов
- **Выводы и рекомендации** — Итоговые рекомендации по выбору сервера

🔗 Шаг 06:

Миссия 6

📄 Задача: Сопоставьте тип графика с его целевым использованием в отчете.

Понятия:

Ответ:

- Столбчатая диаграмма
- Линейный график
- Диаграмма рассеяния
- Таблица с цветовой маркировкой

Определения:

1. Сравнение показателей между серверами
2. Демонстрация трендов во времени
3. Визуализация зависимости двух переменных
4. Показывает отдельные значения с цветовой категоризацией

🔗 Подсказка:

Подумайте, какой график лучше всего показывает тренды, сравнение и зависимости.

- **Столбчатая диаграмма** — Сравнение показателей между серверами
- **Линейный график** — Демонстрация трендов во времени
- **Диаграмма рассеяния** — Визуализация зависимости двух переменных
- **Таблица с цветовой маркировкой** — Показывает отдельные значения с цветовой категоризацией

🔗 Шаг 07:

Миссия 7

Расположите шаги подготовки отчета о выборе сервера в правильном порядке:

1. Сбор данных бенчмарков

2. Выбор метрик и графиков для отчета
3. Создание таблиц и диаграмм
4. Написание выводов и рекомендаций

 Подсказка:

Сначала собираем данные, затем визуализируем и только потом делаем выводы.

Ответ:

- Сбор данных бенчмарков
- Выбор метрик и графиков для отчета
- Создание таблиц и диаграмм
- Написание выводов и рекомендаций

 Шаг 08:

Миссия 8

Расположите шаги анализа данных бенчмарка для включения в отчет:

1. Очистка и подготовка данных
2. Сравнение разных серверов по ключевым метрикам
3. Построение графиков и таблиц
4. Вывод наглядных рекомендаций

 Подсказка:

Сначала обрабатываем данные, потом сравниваем, визуализируем и делаем выводы.

Ответ:

- Очистка и подготовка данных
- Сравнение разных серверов по ключевым метрикам
- Построение графиков и таблиц
- Вывод наглядных рекомендаций

 Шаг 09:

Миссия 9

Анна хочет включить в отчет комментарии по каждому серверу. Как правильно это сделать?

Подсказка: Комментарии должны пояснять результаты, а не повторять цифры графиков.

Ответ:

- **Краткие пояснения по сильным и слабым сторонам сервера**
- Просто копировать все значения из таблиц
- Добавлять личные впечатления без данных
- Писать только цвет корпуса сервера

 Шаг 10:

Миссия 10

Дмитрий планирует включить в отчет сравнительную таблицу серверов. Что важно учесть?

Подсказка: Сравнение должно быть наглядным и по ключевым метрикам, а не по случайным характеристикам.

Ответ:

- **Включить только важные показатели производительности**
- Добавлять все параметры BIOS и процессора

- Сравнивать только внешний вид сервера
- Игнорировать результаты бенчмарков

Шаг 11:

Миссия 11

 Задача: Сопоставьте элемент отчета с его описанием.

Понятия:

Ответ:

- Заголовки разделов
- Графики и диаграммы
- Комментарии к результатам
- Приложения

Определения:

1. Обозначение структуры отчета для удобства навигации
2. Визуализация данных для наглядного сравнения серверов
3. Пояснения и интерпретация метрик
4. Дополнительные таблицы и подробные бенчмарки

Подсказка:

Подумайте, что помогает читать отчет, что показывает данные и где хранятся детали.

- **Заголовки разделов** — Обозначение структуры отчета для удобства навигации
- **Графики и диаграммы** — Визуализация данных для наглядного сравнения серверов
- **Комментарии к результатам** — Пояснения и интерпретация метрик
- **Приложения** — Дополнительные таблицы и подробные бенчмарки

Шаг 12:

Миссия 12

 Задача: Сопоставьте элемент отчета с его функцией.

Понятия:

Ответ:

- Выводы
- Резюме
- Методика тестирования
- Приложения

Определения:

1. Краткое изложение ключевых результатов отчета
2. Подробные шаги, как проводились тесты и нагрузка
3. Итоговые рекомендации по выбору сервера
4. Дополнительные таблицы, графики и данные

Подсказка:

Сначала кратко суммируем результаты, потом описываем методику и добавляем дополнительные материалы.

- **Выводы** — Итоговые рекомендации по выбору сервера
- **Резюме** — Краткое изложение ключевых результатов отчета

- **Методика тестирования** — Подробные шаги, как проводились тесты и нагрузка
- **Приложения** — Дополнительные таблицы, графики и данные

Шаг 13:

Миссия 13

Расположите шаги подготовки визуализаций для отчета:

1. Выбор метрик для отображения
2. Очистка и фильтрация данных
3. Создание графиков и диаграмм
4. Проверка на корректность и читаемость

Подсказка:

Сначала определяем, что показываем, потом подготавливаем данные, визуализируем и проверяем.

Ответ:

- Выбор метрик для отображения
- Очистка и фильтрация данных
- Создание графиков и диаграмм
- Проверка на корректность и читаемость

Шаг 14:

Миссия 14

Расположите шаги подготовки раздела с рекомендациями в отчете:

1. Обобщение результатов тестов
2. Определение сильных и слабых сторон каждого сервера
3. Формулировка рекомендаций для разных сценариев нагрузки
4. Проверка на логичность и ясность формулировок

Подсказка:

Сначала анализируем данные, потом формируем рекомендации и проверяем текст.

Ответ:

- Обобщение результатов тестов
- Определение сильных и слабых сторон каждого сервера
- Формулировка рекомендаций для разных сценариев нагрузки
- Проверка на логичность и ясность формулировок

Шаг 15:

Миссия 15

Иван хочет оформить отчет в виде презентации для руководства. Что важно учесть?

Подсказка: Презентация должна быть наглядной и краткой, не перегружать данными.

Ответ:

- **Использовать наглядные графики и краткие комментарии**
- Копировать все таблицы из отчета
- Добавлять длинные тексты методик
- Игнорировать результаты тестов

Шаг 16:

Миссия 16

Мария хочет, чтобы отчет был понятен как техническим специалистам, так и менеджерам. Что поможет достичь этого?

Подсказка: Используйте наглядные визуализации и простые пояснения без лишних деталей.

Ответ:

- **Комбинация графиков, таблиц и кратких комментариев**
- Только текст без графиков
- Только сложные технические диаграммы
- Игнорировать потребности разных аудиторий

Урок 3 - Практическое тестирование и анализ

Шаг 01:

Миссия 1

Андрей запускает нагрузочное тестирование PostgreSQL на двух серверах: один с SSD, другой с HDD. На что в первую очередь он должен обратить внимание при анализе результатов?

Подсказка: Вспомните, что скорость дисковой подсистемы особенно важна для операций записи и чтения.

Ответ:

- **Сравнить задержки и количество операций ввода-вывода**
- Сравнить только тактовую частоту процессора
- Оценить количество доступных сетевых интерфейсов
- Сравнить только объём памяти

Шаг 02:

Миссия 2

Марина тестирует два варианта сервера. Один имеет больше ядер, другой — выше частоту. Какой параметр чаще оказывается решающим для OLTP-нагрузки?

Подсказка: В OLTP запросы короткие и выполняются параллельно многими пользователями.

Ответ:

- **Количество ядер**
- Частота одного ядра
- Размер кэша процессора
- Поддержка AVX-инструкций

Шаг 03:

Миссия 3

Сергей решил протестировать разные размеры `shared_buffers`. Что он должен проверить в первую очередь при изменении этого параметра?

Подсказка: Этот параметр напрямую влияет на то, как часто сервер обращается к диску.

Ответ:

- **Частоту обращений к дисковой подсистеме**
- Количество сетевых соединений
- Среднюю загрузку процессора

- Число индексов в базе данных

🔗 Шаг 04:

Миссия 4

Елена проводит нагрузочные тесты и замечает рост времени отклика при увеличении числа соединений. Что это может означать?

Подсказка: Подумайте о том, что ресурсы сервера не бесконечны, и параллелизм имеет предел.

Ответ:

- **Сервер достиг предела масштабируемости**
- Произошёл сбой в сети
- Драйвер базы данных устарел
- Запросы были оптимизированы слишком сильно

🔗 Шаг 05:

Миссия 5

Команда решила сравнить результаты `rgbench` на двух серверах. На что нужно обратить внимание при запуске теста?

Подсказка: Думайте о том, как избежать случайных колебаний и получить честное сравнение.

Ответ:

- **Количество прогонов и усреднение результатов**
- Версия Python в системе
- Наличие SSH-доступа
- Размер логов в MB

🔗 Шаг 06:

Миссия 6

Сопоставьте тип теста с его основной целью.

Подсказка: Вспомните, что разные тесты бывают для проверки пиковых нагрузок, стабильности или производительности под реальной нагрузкой.

Ответ:

- **Stress test** — Определение предела возможностей системы при экстремальной нагрузке.
- **Load test** — Измерение производительности при типичной нагрузке.
- **Endurance test** — Проверка стабильности при длительной работе.
- **Benchmark** — Сравнение производительности разных конфигураций.

🔗 Шаг 07:

Миссия 7

Сопоставьте метрику с её значением для анализа.

Подсказка: Думайте о том, что `latency` измеряет время, `throughput` — количество, а `CPU usage` — загрузку процессора.

Ответ:

- **Latency** — Среднее время отклика на запрос.
- **Throughput** — Количество транзакций в секунду.
- **CPU usage** — Процент использования процессора.
- **IOPS** — Количество операций ввода-вывода в секунду.

🔮 Шаг 08:

Миссия 8

Сопоставьте инструмент с его назначением.

Подсказка: Одни инструменты встроены в PostgreSQL, другие внешние.

Ответ:

- **pg_stat_activity** — Отслеживание текущих выполняемых запросов.
- **pg_stat_io** — Сбор статистики по вводу-выводу.
- **pgbench** — Генерация тестовой нагрузки.
- **pgBadger** — Анализ логов PostgreSQL.

🔮 Шаг 09:

Миссия 9

Сопоставьте параметр конфигурации с его ролью.

Подсказка: Подумайте, что отвечает за память, что за контроль ввода-вывода, а что за число процессов.

Ответ:

- **work_mem** — Объём памяти для сортировок и хэшей.
- **max_connections** — Максимальное количество одновременных подключений.
- **checkpoint_timeout** — Интервал между контрольными точками.
- **effective_cache_size** — Оценка доступной ОС дисковой кэш-памяти.

🔮 Шаг 10:

Миссия 10

Сопоставьте тип нагрузки с наиболее уязвимым ресурсом.

Подсказка: Для аналитических запросов обычно узким местом становится диск, для транзакций — CPU и память.

Ответ:

- **OLTP** — Чаще всего упирается в CPU и количество соединений.
- **OLAP** — Чаще всего упирается в диск и операции чтения.
- **Смешанная нагрузка** — Может перегружать и CPU, и диск одновременно.
- **ETL** — Упирается в скорость последовательной записи.

🔮 Шаг 11:

Миссия 11

Сопоставьте команду DBA с её целью.

Подсказка: Подумайте, какие шаги нужны для анализа результатов и выбора оптимальной конфигурации.

Ответ:

- **Собрать метрики** — Получить данные о производительности.
- **Сравнить результаты** — Определить лучший сервер.
- **Проанализировать узкие места** — Найти ресурс, который ограничивает скорость.
- **Сделать рекомендации** — Предложить оптимальную конфигурацию.

🔗 Шаг 12:

Миссия 12

Сопоставьте показания мониторинга с возможной проблемой.

Подсказка: Подумайте, какой ресурс перегружен при высоком latency или при постоянных checkpoint.

Ответ:

- **Высокий CPU usage** — Сервер перегружен вычислениями.
- **Частые checkpoint** — Неправильно настроен checkpoint_timeout.
- **Большие очереди диска** — IO подсистема не справляется.
- **Высокий latency при низкой загрузке** — Возможно, проблема в блокировках или планировщике.

🔗 Шаг 13:

Миссия 13

Расположите шаги проведения нагрузочного теста в правильном порядке.

Подсказка: Сначала идёт подготовка базы, потом прогон тестов, затем сбор метрик и анализ.

Ответ:

- Подготовка базы данных
- Запуск теста
- Сбор метрик
- Анализ результатов

🔗 Шаг 14:

Миссия 14

Расставьте по порядку шаги выбора конфигурации сервера.

Подсказка: Сначала нужно определить задачу, потом провести тесты, затем сравнить и выбрать.

Ответ:

- Определение сценария нагрузки
- Проведение тестов
- Сравнение результатов
- Выбор оптимального сервера

🔗 Шаг 15:

Миссия 15

Расположите шаги работы с rgbench в логическом порядке.

Подсказка: Думайте о том, что сначала нужно инициализировать данные, затем запустить, после чего обработать результаты.

Ответ:

- Инициализация данных
- Запуск теста
- Сбор статистики
- Интерпретация результатов

Шаг 16:

Миссия 16

Расположите этапы анализа узкого места в системе.

Подсказка: Сначала нужно заметить проблему, потом собрать данные, затем определить ресурс и только потом предлагать решение.

Ответ:

- Выявление проблемы
- Сбор метрик
- Определение узкого места
- Выработка рекомендаций

Шаг 17:

Миссия 17

Расставьте шаги подготовки отчёта о бенчмаркинге.

Подсказка: Сначала собираются данные, потом формируются графики, затем пишется текст и делается вывод.

Ответ:

- Сбор данных
- Подготовка графиков
- Написание текста
- Формулировка выводов

Модуль 7 - Продвинутые темы

Урок 1 - Параллелизация запросов в PostgreSQL

Шаг 01:

Миссия 1

Разработчик Антон впервые столкнулся с параллелизацией запросов в PostgreSQL. Какое условие должно быть выполнено, чтобы СУБД могла распараллелить выполнение SELECT-запроса?

Подсказка: Подумайте о размере таблицы и сложности операции — для мелких данных выгода будет минимальной.

Ответ:

- **Запрос должен обрабатывать большое количество строк**
- Запрос всегда распараллеливается автоматически
- Запрос должен выполняться только с одной строкой
- Параллелизация включена по умолчанию даже для мелких таблиц

Шаг 02:

Миссия 2

Программистка Катя тестирует сложный аналитический запрос. Какие параметры PostgreSQL напрямую влияют на количество рабочих процессов для параллельного плана?

Подсказка: Один параметр задаёт максимум воркеров, другой — масштаб параллелизма.

Ответ:

- **max_parallel_workers_per_gather**
- **parallel_setup_cost**
- work_mem
- effective_cache_size

🔮 **Шаг 03:**

Миссия 3

Инженер Сергей заметил, что его запрос не распараллелился. Что может быть причиной?

Подсказка: Проверьте ограничения параллельного плана: не все типы запросов поддерживаются.

Ответ:

- **Запрос содержит функцию, не помеченную как parallel safe**
- Запрос всегда распараллеливается, это ошибка СУБД
- Запрос слишком большой
- Установлен высокий work_mem

🔮 **Шаг 04:**

Миссия 4

Разработчица Лена проверяет EXPLAIN-план. Какой узел плана указывает на использование параллельного выполнения?

Подсказка: Внимательно посмотрите на ключевые слова в плане выполнения.

Ответ:

- **Parallel Seq Scan**
- Index Scan
- Bitmap Heap Scan
- Nested Loop

🔮 **Шаг 05:**

Миссия 5

Программист Дима хочет ускорить отчёт, но видит только один поток выполнения. Какое действие стоит предпринять?

Подсказка: Подумайте о конфигурации и размере данных.

Ответ:

- **Увеличить значение max_parallel_workers_per_gather**
- Уменьшить shared_buffers
- Добавить индексы к каждой колонке
- Отключить параллелизацию

🔮 **Шаг 06:**

Миссия 6

Аналитик Олег хочет понять, какие операции особенно хорошо масштабируются в параллельных планах. Что это за операции?

Подсказка: Вспомните операции, где можно делить набор данных на части и обрабатывать независимо.

Ответ:

- **Seq Scan больших таблиц**
- **Агрегации SUM/AVG**
- INSERT во множество строк
- DELETE с подзапросами

🔗 **Шаг 07:**

Миссия 7

Марина пишет запрос с JOIN. Как JOIN влияет на возможность параллельного плана?

Подсказка: Не все типы соединений одинаково дружат с параллельностью.

Ответ:

- **Hash Join может выполняться параллельно**
- Nested Loop всегда распараллеливается
- Merge Join блокирует параллельность
- JOIN никогда не поддерживает параллелизацию

🔗 **Шаг 08:**

Миссия 8

Разработчик Павел удивился: несмотря на огромную таблицу, запрос не стал параллельным. Почему?

Подсказка: Проверьте настройки стоимостей запуска параллельного плана.

Ответ:

- **parallel_setup_cost слишком высокое**
- shared_buffers слишком маленький
- effective_cache_size слишком большой
- autovacuum выключен

🔗 **Шаг 09:**

Миссия 9

Сисадмин Артём хочет проверить нагрузку. Какая команда в PostgreSQL позволяет увидеть, сколько воркеров выделено для параллельного плана?

Подсказка: Подумайте о выводе EXPLAIN.

Ответ:

- **EXPLAIN (ANALYZE) SELECT ...**
- VACUUM ANALYZE
- SHOW parallel_workers
- pg_stat_activity

🔗 **Шаг 10:**

Миссия 10

📖 **Задача:** Сопоставьте параметры PostgreSQL с их назначением.

Подсказка: Одни отвечают за количество воркеров, другие — за стоимость запуска.

Ответ:

- **max_parallel_workers** — Общее количество воркеров для всей СУБД

- **max_parallel_workers_per_gather** — Максимальное число воркеров для одного запроса
- **parallel_setup_cost** — Стоимость инициализации параллельного плана
- **parallel_tuple_cost** — Стоимость передачи строки между воркерами

🔗 Шаг 11:

Миссия 11

Сопоставьте типы операций с их параллельной поддержкой в PostgreSQL.

Подсказка: Вспомните, какие операции можно делить между воркерами.

Ответ:

- **Parallel Seq Scan** — Чтение строк таблицы параллельно несколькими процессами
- **Parallel Index Scan** — Чтение индекса параллельно (ограниченная поддержка)
- **Parallel Hash Join** — Выполнение соединения через хеш в параллели
- **Parallel Aggregate** — Распределённый подсчёт агрегатных функций

🔗 Шаг 12:

Миссия 12

📄 Сопоставьте роли специалистов с их задачами при настройке параллельных запросов.

Подсказка: Администратор отвечает за конфигурацию, разработчик — за код.

Ответ:

- **DBA** — Настройка параметров max_parallel_workers и стоимости
- **Разработчик** — Написание запросов с учётом parallel safe функций
- **Аналитик** — Проверка отчётов на производительность
- **DevOps** — Мониторинг нагрузки на сервер

🔗 Шаг 13:

Миссия 13

Сопоставьте сценарий с результатом параллелизации.

Подсказка: Подумайте, когда выгода максимальна.

Ответ:

- **Запрос к таблице на 1 млн строк без фильтров** — Сильное ускорение при параллели
- **Запрос к маленькой таблице 100 строк** — Выгода минимальна
- **Запрос с функцией parallel unsafe** — Параллель не используется
- **Запрос с SUM по большой таблице** — Ускорение благодаря разделению набора данных

🔗 Шаг 14:

Миссия 14

Сопоставьте команды PostgreSQL с их назначением при диагностике параллельных запросов.

Подсказка: Вспомните про EXPLAIN и статистические представления.

Ответ:

- **EXPLAIN (ANALYZE)** — Показывает фактический план с количеством воркеров
- **pg_stat_activity** — Показывает текущие выполняющиеся запросы
- **pg_stat_statements** — Анализирует самые ресурсоёмкие запросы
- **VACUUM ANALYZE** — Обновляет статистику для планировщика

Шаг 15:

Миссия 15

Расположите шаги настройки параллельных запросов в PostgreSQL в правильном порядке:

Подсказка: Сначала нужно включить параметры, потом проверить план.

Ответ:

- Установить `max_parallel_workers_per_gather`
- Выполнить EXPLAIN для проверки плана
- Запустить тестовый запрос
- Настроить `parallel_setup_cost`

Урок 2 - Профилирование и мониторинг

Шаг 01:

Миссия 1

Андрей запускает `pg_stat_activity`, чтобы понять, какие запросы сейчас выполняются. Какой ключевой столбец помогает определить, сколько времени запрос выполняется?

Подсказка: Подумайте: важно не только видеть запрос, но и знать его длительность.

Ответ:

- **query_start**
- datid
- state
- username

Шаг 02:

Миссия 2

Марина запускает `pg_stat_statements` и хочет узнать, какие запросы чаще всего потребляют CPU. Какой показатель стоит смотреть?

Подсказка: Речь идёт о времени работы процессора, а не о диске или памяти.

Ответ:

- **total_exec_time**
- rows
- calls
- mean_time

Шаг 03:

Миссия 3

Дмитрий запускает EXPLAIN (ANALYZE) для запроса. Какой параметр поможет понять, сколько строк реально было возвращено оператором?

Подсказка: Подумайте, чем отличается планируемое количество строк от фактического.

Ответ:

- **Actual Rows**
- Estimated Rows
- Startup Cost
- Parallel Workers

🔧 Шаг 04:

Миссия 4

Иван хочет мониторить загрузку диска PostgreSQL при нагрузочном тестировании. Какой инструмент в Linux чаще всего используют для этого?

Подсказка: Этот инструмент показывает IOPS, пропускную способность и задержки.

Ответ:

- **iostat**
- ps
- netstat
- htop

🔧 Шаг 05:

Миссия 5

Ольга анализирует slow query log и видит запрос, выполняющийся 30 секунд. Какой параметр конфигурации отвечает за порог записи медленных запросов?

Подсказка: Это настройка с суффиксом `_time`.

Ответ:

- **log_min_duration_statement**
- log_statement
- track_io_timing
- log_lock_waits

🔧 Шаг 06:

Миссия 6

Петр хочет узнать текущую активность по блокировкам в PostgreSQL. Какое системное представление ему стоит использовать?

Подсказка: В названии есть слово `locks`.

Ответ:

- **pg_locks**
- pg_stat_user_tables
- pg_stat_activity
- pg_stat_database

🔧 Шаг 07:

Миссия 7

Алексей хочет замерить потребление памяти PostgreSQL. Какой инструмент Linux подходит лучше всего?

Подсказка: Этот инструмент показывает распределение памяти процессами.

Ответ:

- **smem**
- ping
- iftop
- iotop

🔗 Шаг 08:

Миссия 8

Сопоставьте метрику с её назначением при профилировании PostgreSQL.

Подсказка: Вспомните, что одни метрики связаны с диском, другие — с процессором или памятью.

Ответ:

- **pg_stat_activity** — Показывает активные запросы и их состояние
- **pg_stat_io** — Информация об операциях ввода-вывода
- **pg_stat_database** — Статистика по базам данных: количество транзакций, время запросов
- **pg_stat_bgwriter** — Метрики фоновой записи буферов

🔗 Шаг 09:

Миссия 9

Сопоставьте инструмент мониторинга с его описанием.

Подсказка: Обратите внимание: одни утилиты встроены в Linux, другие — специфичны для PostgreSQL.

Ответ:

- **pg_stat_statements** — Анализ часто выполняемых запросов и их времени
- **iostat** — Показывает нагрузку на дисковую подсистему
- **vmstat** — Информация о памяти, процессоре и очередях
- **pg_locks** — Отслеживание текущих блокировок

🔗 Шаг 10:

Миссия 10

Сопоставьте pgbench-параметр с его назначением.

Подсказка: В названии ключа обычно скрыт намёк на его использование.

Ответ:

- **-c** — Количество параллельных клиентов
- **-T** — Длительность теста
- **-j** — Количество рабочих потоков
- **-n** — Игнорировать время подготовки

🔗 Шаг 11:

Миссия 11

Сопоставьте pg_stat_database метрику с её значением.

Подсказка: Вспомните: одни поля про транзакции, другие — про задержки и блокировки.

Ответ:

- **numbackends** — Количество подключённых клиентов
- **xact_commit** — Число зафиксированных транзакций
- **deadlocks** — Количество обнаруженных взаимных блокировок
- **blks_hit** — Количество обращений к буферному кэшу

Шаг 12:

Миссия 12

Расположите шаги анализа медленного запроса в PostgreSQL в правильном порядке.

Подсказка: Сначала ищем проблемный запрос, затем анализируем план, потом оптимизируем.

Ответ:

- Найти запрос в логах или `pg_stat_statements`
- Выполнить `EXPLAIN (ANALYZE)`
- Определить узкие места (индексы, фильтры)
- Внести оптимизацию (индексы, переписывание запроса)

Шаг 13:

Миссия 13

Упорядочьте действия по настройке мониторинга PostgreSQL.

Подсказка: Сначала включается расширение, затем собираются данные, потом анализ и настройка алертов.

Ответ:

- Установить и включить `pg_stat_statements`
- Собрать статистику за период
- Проанализировать проблемные запросы
- Настроить оповещения

Шаг 14:

Миссия 14

Расположите шаги проведения нагрузочного теста в `pgbench`.

Подсказка: Сначала инициализация, потом запуск, затем анализ.

Ответ:

- Инициализировать базу данных для теста
- Запустить тест с параметрами
- Собрать статистику выполнения
- Сделать выводы и оптимизацию

Шаг 15:

Миссия 15

Расставьте этапы настройки алертов в системе мониторинга PostgreSQL.

Подсказка: Логика проста: сбор метрик → определение порогов → настройка алертов → проверка.

Ответ:

- Собрать ключевые метрики (CPU, IO, запросы)
- Определить критические пороговые значения
- Настроить правила оповещений
- Проверить работу алертов на тестовых нагрузках

Урок 3 - Оценка производительности на больших данных

Шаг 01:

Миссия 1

Андрей запускает тесты на большом наборе данных и замечает, что индексы сильно влияют на скорость запросов. Какой тип индекса чаще всего эффективен при работе с текстовыми полями для поиска по подстроке?

Подсказка: Вспомни индексы, оптимизированные для полнотекстового и LIKE-запросов.

Ответ:

- **GIN-индекс**
- B-Tree индекс
- Hash индекс
- BRIN индекс

Шаг 02:

Миссия 2

Марина анализирует нагрузку на сервер и видит большое количество последовательных сканирований таблицы в 200 млн строк. Какое решение может сократить затраты на диск и ускорить выполнение запросов?

Подсказка: Подумай о типе индексов, которые полезны при работе с большими, отсортированными наборами данных.

Ответ:

- **Использование BRIN индекса**
- Добавление уникального ограничения
- Перевод таблицы в JSONB
- Удаление индексов

Шаг 03:

Миссия 3

Олег делает бенчмаркинг сложного отчётного запроса. Он замечает, что при большом объёме данных планировщик выбирает последовательное сканирование вместо использования индекса. Что может быть причиной?

Подсказка: Планировщик выбирает стратегию, если она дешевле по его оценке.

Ответ:

- **Низкая селективность запроса**
- Ошибки в SQL синтаксисе
- Неверный формат даты
- Недостаток памяти на сервере

Шаг 04:

Миссия 4

Иван настроил кластер PostgreSQL для анализа логов в 1 ТБ. Какой инструмент встроен в PostgreSQL для просмотра плана выполнения запросов?

Подсказка: Используется ключевое слово перед SQL-запросом.

Ответ:

- **EXPLAIN**
- PROFILE
- DESCRIBE
- TRACE

🔮 **Шаг 05:**

Миссия 5

Светлана проводит стресс-тестирование и видит, что узким местом стали дисковые операции. Какое решение поможет снизить нагрузку на диск при работе с большими таблицами?

Подсказка: Подумай о хранении только нужных данных в активных запросах.

Ответ:

- **Партиционирование таблиц**
- Удаление всех индексов
- Использование UUID вместо SERIAL
- Сжатие логов вручную

🔮 **Шаг 06:**

Миссия 6

📖 Сопоставьте виды индексов PostgreSQL с их назначением.

Подсказка: Вспомни: B-Tree — универсален, GIN — текст и массивы, BRIN — большие диапазоны.

Ответ:

- **B-Tree** — Оптimalен для сравнения чисел и сортировки
- **GIN** — Эффективен для полнотекстового поиска и массивов
- **BRIN** — Экономичен для больших таблиц с диапазонами
- **Hash** — Быстрый поиск по точному совпадению

🔮 **Шаг 07:**

Миссия 7

📖 Сопоставьте команды PostgreSQL с их назначением для анализа производительности.

Подсказка: Некоторые команды показывают статистику, другие — структуру плана.

Ответ:

- **EXPLAIN** — Показывает план выполнения запроса
- **EXPLAIN ANALYZE** — Запускает запрос и выводит реальные метрики
- **VACUUM** — Очищает и оптимизирует таблицу
- **ANALYZE** — Обновляет статистику о распределении данных

🔮 **Шаг 08:**

Миссия 8

📖 Соотнесите показатели мониторинга с их смыслом.

Подсказка: Важно понимать разницу между загрузкой процессора, диска и памяти.

Ответ:

- **CPU usage** — Процент времени процессора, занятый запросами
- **I/O wait** — Время ожидания операций чтения и записи

- **Shared buffers** — Объем памяти для кэширования страниц
- **Checkpoint** — Сброс изменённых страниц на диск

🔧 Шаг 09:

Миссия 9

📖 Соотнесите инструменты мониторинга PostgreSQL с их назначением.

Подсказка: Один инструмент встроен, другие внешние.

Ответ:

- **pg_stat_activity** — Показывает текущие запросы и соединения
- **pg_stat_statements** — Собирает статистику по выполненным запросам
- **pgAdmin** — Графическая админка с мониторингом
- **Prometheus** — Сбор метрик для визуализации и алертов

🔧 Шаг 10:

Миссия 10

📖 Соотнесите тип нагрузки с подходящим методом оптимизации.

Подсказка: Вспомни OLTP и OLAP.

Ответ:

- **OLTP (много коротких транзакций)** — Нужны быстрые индексы и минимальная блокировка
- **OLAP (аналитика больших данных)** — Партиционирование и агрегация
- **Высокая нагрузка на запись** — Использование batch-вставок и COPY
- **Высокая нагрузка на чтение** — Кэширование и репликация

🔧 Шаг 11:

Миссия 11

📖 Соотнесите типы партиционирования с их описанием.

Подсказка: Один из вариантов — диапазон, другой — список.

Ответ:

- **Партиционирование по диапазону** — Разделение таблицы на основе интервалов значений
- **Партиционирование по списку** — Разделение по конкретным значениям ключа
- **Партиционирование по хэшу** — Равномерное распределение по хэш-функции
- **Секционирование вручную** — Создание отдельных таблиц без автоматического управления

🔧 Шаг 12:

Миссия 12

📖 Соотнесите настройки PostgreSQL с их влиянием.

Подсказка: shared_buffers отвечает за память, work_mem — за операции сортировки.

Ответ:

- **shared_buffers** — Размер кэша страниц
- **work_mem** — Память для сортировок и хеш-таблиц
- **maintenance_work_mem** — Память для VACUUM и CREATE INDEX
- **effective_cache_size** — Оценка доступного кэша ОС

Шаг 13:

Миссия 13

Расположите шаги анализа запроса в правильном порядке:

Подсказка: Сначала план, потом запуск, потом оптимизация.

Ответ:

- Выполнить EXPLAIN
- Запустить EXPLAIN ANALYZE
- Определить узкие места
- Оптимизировать запрос

Шаг 14:

Миссия 14

Упорядочите этапы настройки сервера для больших данных:

Подсказка: Сначала базовые параметры, потом оптимизация памяти, затем индексы.

Ответ:

- Настроить shared_buffers
- Настроить work_mem
- Добавить индексы
- Включить логирование медленных запросов

Шаг 15:

Миссия 15

Расположите действия по оптимизации больших join-запросов в правильном порядке:

Подсказка: Индексы → анализ плана → проверка статистики.

Ответ:

- Создать индексы на ключевых колонках
- Выполнить EXPLAIN
- Проверить актуальность ANALYZE
- Измерить время выполнения

Шаг 16:

Миссия 16

Упорядочите шаги масштабирования PostgreSQL:

Подсказка: Репликация идёт после оптимизации одного сервера.

Ответ:

- Настроить параметры памяти
- Добавить индексы и партиционирование
- Включить репликацию
- Использовать балансировщик нагрузки

Шаг 17:

Миссия 17

Расположите шаги бенчмаркинга PostgreSQL на больших данных в правильной последовательности:

Подсказка: Подготовка → запуск тестов → сбор метрик → анализ.

Ответ:

- Подготовить тестовые данные
- Запустить нагрузочные тесты
- Собрать статистику
- Сделать выводы и оптимизировать

Модуль 8 - Бонус. Заключение и дальнейшее развитие

Урок 1 - Итоговое задание

Шаг 01:

Миссия 1

Вы работаете в команде, которая завершает проект по нагрузочному тестированию PostgreSQL. Какой первый шаг при подготовке итогового отчёта о производительности?

Подсказка: Начните с общего обзора того, что именно тестировалось, чтобы у читателя сложилась картина.

Ответ:

- **Описание целей и сценариев нагрузочного тестирования**
- Сразу таблица с графиками без пояснений
- Только выводы без фактов
- Список участников команды

Шаг 02:

Миссия 2

Анна анализирует отчёты по бенчмаркам. Она заметила, что время отклика выросло при увеличении числа подключений. Как правильно это зафиксировать в итоговом отчёте?

Подсказка: Итоговый отчёт ценен тем, что связывает метрики с факторами нагрузки.

Ответ:

- **Связать рост времени отклика с увеличением количества подключений**
- Записать: «результат странный»
- Не упоминать это в отчёте
- Описать только хостинг без цифр

Шаг 03:

Миссия 3

Влад хочет сделать заключение о масштабируемости сервера. На какие данные ему стоит опираться?

Подсказка: Масштабируемость — это поведение под увеличением нагрузки.

Ответ:

- **Сравнение throughput и latency при разных уровнях нагрузки**
- Только среднее время отклика на одной нагрузке
- Версию драйвера подключения
- Количество программистов в команде

🔮 Шаг 04:

Миссия 4

При финальном обсуждении Дима предложил включить в отчёт рекомендации. Что должно быть главным?

Подсказка: Рекомендации всегда должны опираться на реальные результаты тестирования.

Ответ:

- **Конкретные настройки PostgreSQL, улучшившие производительность**
- Общие советы без цифр
- Истории из личного опыта без данных
- Только список использованных инструментов

🔮 Шаг 05:

Миссия 5

Света готовит презентацию итогов бенчмаркинга. Что лучше всего использовать для демонстрации динамики изменения производительности?

Подсказка: Визуализация важна для того, чтобы руководители быстро уловили тренды.

Ответ:

- **Графики зависимости нагрузки от времени отклика**
- Сырые SQL-логи
- Список подключений в текстовом файле
- Только устный пересказ без слайдов

🔮 Шаг 06:

Миссия 6

Сопоставьте ключевые разделы итогового отчёта с их содержанием.

Подсказка: Подумайте о логике: от постановки задачи до выводов.

Ответ:

- **Введение** — Описание целей и контекста тестирования
- **Методология** — Описание инструментов и сценариев нагрузок
- **Результаты** — Таблицы и графики производительности
- **Заключение** — Выводы и рекомендации для оптимизации

🔮 Шаг 07:

Миссия 7

Илья пишет отчёт. Помогите ему сопоставить метрику с тем, что она показывает.

Подсказка: Метрики отвечают на разные вопросы: скорость, стабильность, использование ресурсов.

Ответ:

- **Throughput** — Количество транзакций в секунду
- **Latency** — Время ответа на одну транзакцию
- **CPU Utilization** — Загрузка процессора
- **Disk I/O** — Скорость чтения и записи на диск

🔗 Шаг 08:

Миссия 8

Сопоставьте инструмент мониторинга с его основным назначением.

Подсказка: Каждый инструмент закрывает определённый аспект анализа.

Ответ:

- **pg_stat_statements** — Анализ частоты и стоимости запросов
- **EXPLAIN ANALYZE** — План выполнения и время запроса
- **iostat** — Мониторинг дисковых операций
- **top/htop** — Нагрузка на процессор и память

🔗 Шаг 09:

Миссия 9

Сопоставьте тип нагрузки с её примером.

Подсказка: Нагрузка может быть на чтение, запись или смешанная.

Ответ:

- **Read-heavy** — SELECT-запросы с большим количеством JOIN
- **Write-heavy** — Массовые INSERT и UPDATE операций
- **Mixed workload** — Сочетание чтения и записи
- **Analytical workload** — Сложные запросы с агрегатами и GROUP BY

🔗 Шаг 10:

Миссия 10

Сопоставьте ошибку интерпретации результатов с её правильным описанием.

Подсказка: Ошибки часто связаны с неверными обобщениями или игнорированием деталей.

Ответ:

- **Игнорирование outliers** — Неучёт редких, но критичных пиков задержек
- **Выводы без контекста** — Заключение без учёта условий теста
- **Сравнение разных окружений** — Сопоставление данных с несопоставимых серверов
- **Опора на среднее** — Среднее скрывает распределение задержек

🔗 Шаг 11:

Миссия 11

Сопоставьте роль в команде с её вкладом в итоговое задание.

Подсказка: Каждая роль отвечает за свой кусочек пазла.

Ответ:

- **DevOps-инженер** — Подготовка окружения и серверов
- **Аналитик** — Интерпретация графиков и метрик
- **Разработчик** — Оптимизация запросов
- **Тимлид** — Формирование выводов и представление отчёта

🔗 Шаг 12:

Миссия 12

Расположите шаги подготовки итогового отчёта в правильном порядке.

Подсказка: Сначала собираются данные, потом они анализируются и визуализируются.

Ответ:

- Сбор метрик нагрузочного теста
- Построение графиков и таблиц
- Анализ закономерностей
- Формирование выводов и рекомендаций

Шаг 13:

Миссия 13

Расставьте в правильном порядке этапы анализа большой базы данных.

Подсказка: Подумайте: сначала смотрим на сырые данные, затем отбрасываем шум и строим выводы.

Ответ:

- Извлечение сырых логов
- Фильтрация нерелевантных данных
- Анализ трендов
- Формирование выводов

Шаг 14:

Миссия 14

Определите правильную последовательность шагов при демонстрации итогов заказчику.

Подсказка: Важно вести рассказ от простого к сложному.

Ответ:

- Объяснение целей тестирования
- Показ ключевых графиков
- Обсуждение узких мест
- Предложение решений

Шаг 15:

Миссия 15

Расставьте этапы подготовки презентации для руководства.

Подсказка: Сначала материал нужно структурировать, а затем визуализировать.

Ответ:

- Сбор ключевых выводов
- Выбор подходящих графиков
- Создание слайдов
- Репетиция доклада

Шаг 16:

Миссия 16

Расположите шаги при подготовке рекомендаций для оптимизации сервера.

Подсказка: Сначала определите проблему, потом предложите улучшение.

Ответ:

- Определение узких мест

- Подбор параметров конфигурации
- Тестирование изменений
- Документирование результата

Шаг 17:

Миссия 17

Вы завершаете курс. Расставьте шаги самооценки после финального задания.

Подсказка: Сначала посмотрите на результат, потом оцените процесс.

Ответ:

- Просмотр финального отчёта
- Оценка сильных и слабых сторон
- Определение направлений развития
- Формулировка личных выводов

Урок 2 - Советы по мониторингу и оптимизации в реальных условиях

Шаг 01:

Миссия 1

Разработчик Миша анализирует систему логов PostgreSQL и хочет понять, что чаще всего сигнализирует о проблемах с производительностью.

Подсказка: Подумайте, какие сообщения встречаются при узких местах, связанных с памятью или вводом-выводом.

Ответ:

- **Сообщения об ожидании блокировок и долгих запросах**
- Сообщения о старте сервера
- Записи об успешной аутентификации
- Обычные сообщения о контрольных точках (checkpoint)

Шаг 02:

Миссия 2

Аня заметила, что база под нагрузкой часто пишет данные на диск. Она задала вопрос: какой параметр стоит проверять в первую очередь?

Подсказка: Этот параметр связан с размером памяти для буферов.

Ответ:

- **shared_buffers**
- max_connections
- work_mem
- log_min_duration_statement

Шаг 03:

Миссия 3

Программист Олег выполняет бенчмаркинг запросов и видит, что одни и те же SELECT'ы выполняются очень медленно. Он думает, что причина в отсутствии индексов. Как быстро проверить это?

Подсказка: Подумайте о стандартном инструменте PostgreSQL для анализа плана выполнения.

Ответ:

- **Использовать EXPLAIN (ANALYZE)**
- Открыть файл postgresql.conf
- Запустить VACUUM FULL
- Изменить autovacuum на off

 **Шаг 04:**

Миссия 4

DevOps-инженер Даша мониторит PostgreSQL через pg_stat_activity. Она замечает множество сессий в статусе idle in transaction. Что это значит?

Подсказка: Вспомните, что бывает, если транзакцию открыли, но не завершили.

Ответ:

- **Открытые транзакции простаивают и удерживают ресурсы**
- Сессии в данный момент активно выполняют запросы
- Процессы простаивают после завершения транзакций
- Это нормальный рабочий режим без проблем

 **Шаг 05:**

Миссия 5

Программист Игорь использует pg_stat_statements для анализа нагрузки. Какой показатель поможет понять, какие запросы чаще всего нагружают систему?

Подсказка: Обратите внимание на частоту выполнения запросов.

Ответ:

- **calls**
- rows
- mean_exec_time
- shared_blks_read

 **Шаг 06:**

Миссия 6

 **Задача:** Сопоставьте инструмент мониторинга PostgreSQL с его назначением.

Подсказка: Один инструмент работает с планами запросов, другой — с активными соединениями, третий — со статистикой по SQL.

Ответ:

- **EXPLAIN** — Показ плана выполнения запросов
- **pg_stat_activity** — Мониторинг активных подключений и транзакций
- **pg_stat_statements** — Сбор статистики по SQL-запросам
- **auto_explain** — Автоматическая запись планов выполнения долгих запросов в логи

 **Шаг 07:**

Миссия 7

 **Задача:** Сопоставьте параметр конфигурации с его влиянием на производительность.

Подсказка: Вспомните, что связано с памятью, что с контрольными точками, а что с сортировкой.

Ответ:

- **work_mem** — Размер памяти для сортировок и хеш-таблиц
- **checkpoint_timeout** — Частота контрольных точек
- **shared_buffers** — Размер буфера для кэширования данных
- **maintenance_work_mem** — Память для операций VACUUM и CREATE INDEX

Шаг 08:

Миссия 8

 Задача: Сопоставьте метрику с её правильным толкованием.

Подсказка: Обратите внимание на время выполнения, количество вызовов и количество строк.

Ответ:

- **calls** — Количество вызовов конкретного запроса
- **mean_exec_time** — Среднее время выполнения запроса
- **rows** — Количество возвращённых строк
- **total_exec_time** — Суммарное время выполнения всех вызовов

Шаг 09:

Миссия 9

 Задача: Сопоставьте ситуацию с подходящим инструментом для анализа.

Подсказка: Подумайте: для медленных запросов есть логирование, для нагрузки — pg_stat_statements.

Ответ:

- **Долгие SELECT-запросы** — auto_explain
- **Много соединений в статусе idle** — pg_stat_activity
- **Выявление самых нагружающих SQL** — pg_stat_statements
- **Оптимизация плана сложного запроса** — EXPLAIN (ANALYZE)

Шаг 10:

Миссия 10

 Задача: Сопоставьте тип нагрузки с оптимизацией.

Подсказка: Чтение требует индексов, массовая вставка — правильной настройки autovacuum.

Ответ:

- **Чтение больших таблиц** — Использование индексов
- **Массовые INSERT** — Настройка autovacuum и FILLFACTOR
- **Частые UPDATE** — Индекс по обновляемым полям
- **Сложные JOIN** — Оптимизация work_mem

Шаг 11:

Миссия 11

Расставьте шаги по оптимизации медленного запроса в правильном порядке.

Подсказка: Сначала анализ, потом проверка индексов, затем настройка параметров, и только в конце — рефакторинг SQL.

Ответ:

- Проанализировать план выполнения через EXPLAIN

- Проверить наличие необходимых индексов
- Настроить параметры `work_mem` и `join_collapse_limit`
- Переписать SQL-запрос для упрощения

Шаг 12:

Миссия 12

Расположите шаги анализа нагрузки сервера PostgreSQL в правильной последовательности.

Подсказка: Сначала собираются данные мониторинга, потом ищутся узкие места, затем тестируются гипотезы.

Ответ:

- Собрать статистику через `pg_stat_statements`
- Определить самые тяжёлые запросы
- Проверить настройки памяти и индексов
- Запустить нагрузочное тестирование после изменений

Шаг 13:

Миссия 13

Выберите правильный порядок действий при настройке `autovacuum`.

Подсказка: Сначала определите проблему, потом настройте пороги, и только потом тестируйте.

Ответ:

- Определить таблицы с частыми обновлениями
- Настроить `autovacuum_vacuum_scale_factor`
- Установить `autovacuum_analyze_scale_factor`
- Проверить эффект на производительность

Шаг 14:

Миссия 14

Расположите шаги по выявлению проблем блокировок.

Подсказка: Сначала смотрим активные процессы, затем ищем блокировки, потом анализируем запросы.

Ответ:

- Посмотреть `pg_stat_activity`
- Определить запросы, ожидающие блокировки
- Найти запросы, удерживающие блокировки
- Принять решение: переписать запросы или увеличить ресурсы

Шаг 15:

Миссия 15

Определите порядок шагов для оптимизации большого количества соединений.

Подсказка: Подумайте про `connection pool`, настройки и тесты.

Ответ:

- Проанализировать `pg_stat_activity`
- Установить `connection pooler` (PgBouncer/pgpool-II)
- Настроить `max_connections` и `pool_size`
- Протестировать работу приложения под нагрузкой

Шаг 16:

Миссия 16

Выберите правильный порядок шагов при оптимизации логирования.

Подсказка: Сначала включают логирование медленных запросов, потом настраивают порог, и только потом анализируют.

Ответ:

- Включить `log_min_duration_statement`
- Настроить порог времени для логирования
- Собрать и проанализировать логи
- Оптимизировать выявленные проблемные запросы

Урок 3 - Подведение итогов и дальнейшие шаги

Шаг 01:

Миссия 1

Андрей запускает PostgreSQL для внутренней CRM и замечает рост нагрузки. Какой первый шаг в подведении итогов тестирования он должен сделать?

Подсказка: Начинать всегда стоит с анализа того, что уже собрано в логах и статистике.

Ответ:

- **Проверить статистику выполнения запросов через `pg_stat_statements`**
- Сразу менять конфигурацию `shared_buffers`
- Добавить больше индексов на все таблицы
- Перенести базу на новый сервер без анализа

Шаг 02:

Миссия 2

Оля готовит отчёт о нагрузочном тестировании. Её коллеги спорят: что включить в итоговую таблицу?

Подсказка: Сосредоточьтесь на показателях, отражающих эффективность и стабильность системы.

Ответ:

- **Среднее время отклика запросов**
- **Максимальное использование CPU**
- Количество пользователей в чате компании
- **Количество блокировок и deadlocks**

Шаг 03:

Миссия 3

Команда тестировала PostgreSQL на двух серверах: один с быстрым SSD, другой со старыми HDD. Результаты сильно отличаются. Какое резюме стоит вынести?

Подсказка: Хранилище — критически важный фактор производительности.

Ответ:

- **SSD обеспечивают существенно лучшие результаты под нагрузкой**
- Тип диска не имеет значения
- HDD быстрее благодаря большому объёму

- Результаты не стоит учитывать

Шаг 04:

Миссия 4

После серии нагрузочных тестов Вика хочет сделать вывод о масштабируемости PostgreSQL. Какой показатель лучше всего отразит этот аспект?

Подсказка: Масштабируемость связана с ростом нагрузки и ростом времени ответа.

Ответ:

- **Изменение времени отклика при росте количества параллельных соединений**
- Объём занимаемой памяти
- Размер исходного кода PostgreSQL
- Количество таблиц в схеме

Шаг 05:

Миссия 5

Илья завершил бенчмаркинг и готовит дальнейшие шаги. Что будет самым правильным завершением цикла?

Подсказка: После тестирования важно закрепить знания и подготовить план действий.

Ответ:

- **Документирование результатов и составление рекомендаций**
- Удаление всех логов, чтобы сэкономить место
- Игнорирование полученных данных
- Переустановка ОС на сервере

Шаг 06:

Миссия 6

Сопоставьте итоговые шаги анализа нагрузки с их целью.

Подсказка: Подумайте, какие шаги помогают зафиксировать результаты, а какие — понять глубже детали.

Ответ:

- **Документирование отчёта** — Фиксация результатов тестирования для команды
- **Сравнение серверов** — Выявление оптимального варианта для продакшена
- **Повторное тестирование** — Проверка стабильности выводов
- **Анализ графиков CPU/IO** — Понимание узких мест производительности

Шаг 07:

Миссия 7

Сопоставьте шаги после бенчмаркинга с их практическим эффектом.

Подсказка: Важно видеть, зачем делать каждый шаг — он должен иметь ценность для команды.

Ответ:

- **Презентация результатов менеджменту** — Обоснование бюджета на серверы
- **Сравнение конфигураций PostgreSQL** — Выбор оптимальных параметров для нагрузки
- **Подготовка чек-листа мониторинга** — Упрощение поддержки в будущем
- **Обсуждение выводов в команде** — Формирование коллективного понимания

Шаг 08:

Миссия 8

Сопоставьте выводы нагрузочного теста с правильными интерпретациями.

Подсказка: Один и тот же график может говорить о разном, важно понять контекст.

Ответ:

- **Рост latency при 200+ соединениях** — Сервер перестаёт масштабироваться линейно
- **Снижение TPS при росте нагрузки** — Узкие места в CPU или дисковой системе
- **Высокий уровень блокировок** — Конкуренция транзакций за ресурсы
- **Стабильный отклик при росте до 100 соединений** — Хорошая масштабируемость в пределах теста

Шаг 09:

Миссия 9

Сопоставьте действия программиста с их вкладом в дальнейшие шаги.

Подсказка: Каждый шаг помогает сделать выводы более осознанными.

Ответ:

- **Формирование списка проблемных запросов** — Подготовка задач для оптимизации
- **Сравнение PostgreSQL с альтернативами** — Понимание конкурентных преимуществ
- **Определение пределов масштабируемости** — Формирование границ SLA
- **Создание базы знаний команды** — Сохранение опыта для новых сотрудников

Шаг 10:

Миссия 10

Сопоставьте тип отчёта после теста с его назначением.

Подсказка: Разные адресаты требуют разного уровня детализации.

Ответ:

- **Технический отчёт** — Детали конфигураций, графики, TPS, latency
- **Краткое резюме** — Основные выводы для руководителей
- **Инструкции по настройке** — Пошаговое руководство для админов
- **Презентация для команды** — Совместное обсуждение и планирование

Шаг 11:

Миссия 11

Расположите шаги подведения итогов в правильном порядке:

Подсказка: Сначала нужно собрать всё, потом обработать и только в конце презентовать.

Ответ:

- Сбор логов и статистики
- Анализ метрик и построение графиков
- Формулировка выводов
- Презентация команде

Шаг 12:

Миссия 12

Расположите шаги по подготовке рекомендаций после тестирования:

Подсказка: Сначала фиксируются проблемы, потом ищутся решения.

Ответ:

- Идентификация узких мест
- Поиск возможных решений
- Оценка влияния изменений
- Формирование списка рекомендаций

Шаг 13:

Миссия 13

Расположите шаги по организации базы знаний после бенчмаркинга:

Подсказка: Важно сначала собрать, потом структурировать.

Ответ:

- Сбор всех отчётов и логов
- Выделение ключевых кейсов
- Структурирование по темам
- Публикация в корпоративной wiki

Шаг 14:

Миссия 14

Расположите шаги по внедрению выводов теста в жизнь:

Подсказка: Из анализа рождаются задачи, затем они реализуются.

Ответ:

- Формирование списка оптимизаций
- Создание задач в трекере
- Выполнение оптимизаций
- Повторное тестирование

Шаг 15:

Миссия 15

Расположите шаги по подготовке к следующему этапу развития инфраструктуры:

Подсказка: Всё начинается с анализа текущего состояния.

Ответ:

- Анализ текущих ограничений
- Определение целей развития
- Выбор возможных технологий
- Планирование пилотных проектов

Шаг 16:

Миссия 16

Расположите шаги по созданию культуры постоянного тестирования:

Подсказка: Культура строится через практику.

Ответ:

- Регулярные плановые тесты
- Анализ результатов и обсуждение

- Обновление базы знаний
- Интеграция в процессы разработки

🔗 Шаг 17:

Миссия 17

Расположите шаги по подведению итогов курса:

Подсказка: Вспомните, что любая учёба заканчивается практикой и планами на будущее.

Ответ:

- Повторение ключевых идей
- Выполнение итогового задания
- Обсуждение результатов с наставником
- Определение следующих шагов

📖 Модуль 9 - Бонус. Долгосрочная поддержка и планирование

📖 Урок 1 - Как планировать рост нагрузки на сервер

🔗 Шаг 01:

Миссия 1

Аня — разработчик, её стартап быстро набирает пользователей. Какой первый шаг при планировании роста нагрузки на PostgreSQL-сервер?

Подсказка: Прежде чем думать о масштабировании, важно понимать текущие цифры.

Ответ:

- **Измерить текущую нагрузку и собрать метрики**
- Сразу докупить ещё один сервер
- Оптимизировать SQL-запросы наугад
- Переехать на другую СУБД без анализа

🔗 Шаг 02:

Миссия 2

Иван задумался: что помогает предсказать рост нагрузки при запуске рекламной кампании?

Подсказка: История всегда повторяется.

Ответ:

- **Анализ исторических данных и сезонности**
- Отключение логов для ускорения работы
- Ожидание, пока пользователи сами сообщат о проблемах
- Игнорирование прошлых метрик

🔗 Шаг 03:

Миссия 3

Команда выбрала сервер для PostgreSQL. Какой ресурс чаще всего становится узким местом при росте нагрузки?

Подсказка: Диски быстрые, но что-то тормозит раньше.

Ответ:

- **Оперативная память**
- Количество USB-портов
- Видеокарта
- Сетевая карта на 10 Мбит/с

🔮 Шаг 04:

Миссия 4

Сергей планирует нагрузочное тестирование. Что нужно сделать до запуска стресс-тестов?
Подсказка: Без базы знаний тест будет бесполезен.

Ответ:

- **Собрать базовые метрики в спокойном режиме**
- Закрыть все фоновые процессы
- Сменить язык запросов
- Отключить мониторинг для ускорения

🔮 Шаг 05:

Миссия 5

Мария обсуждает с коллегами, как учесть непредсказуемые всплески нагрузки. Что стоит предусмотреть?

Подсказка: Иногда лучше иметь запас, чем сидеть в ночи на аварии.

Ответ:

- **Планирование резервов ресурсов и горизонтального масштабирования**
- Отключение резервного копирования
- Переход на устаревшую версию PostgreSQL
- Игнорирование аномалий

🔮 Шаг 06:

Миссия 6

Сопоставьте показатели мониторинга с их значением при планировании роста нагрузки.

Подсказка: Каждый параметр даёт свой сигнал о будущем.

Ответ:

- **TPS (Transactions per Second)** — Скорость выполнения транзакций в секунду
- **I/O Wait** — Время ожидания операций чтения/записи
- **Cache hit ratio** — Доля запросов, обслуженных из кэша
- **Latency** — Задержка ответа на запрос

🔮 Шаг 07:

Миссия 7

Команда распределяет роли для планирования роста нагрузки. Сопоставьте задачу с ответственным.

Подсказка: DBA не пишет маркетинг, а DevOps не пишет тексты.

Ответ:

- **DBA** — Настройка параметров PostgreSQL и индексов

- **DevOps** — Организация мониторинга и CI/CD
- **Аналитик** — Прогнозирование пиков нагрузки
- **Маркетолог** — Планирование рекламных кампаний

🔮 Шаг 08:

Миссия 8

Сопоставьте подходы к масштабированию PostgreSQL с их описанием.

Подсказка: Один способ увеличивает мощности, другой — количество узлов.

Ответ:

- **Вертикальное масштабирование** — Увеличение ресурсов одного сервера
- **Горизонтальное масштабирование** — Добавление новых серверов
- **Шардинг** — Разделение данных на части
- **Репликация** — Создание копий данных на других узлах

🔮 Шаг 09:

Миссия 9

Сопоставьте инструмент с его использованием при прогнозе роста нагрузки.

Подсказка: Мониторинг — это глаза, нагрузочное тестирование — тренировка.

Ответ:

- **pg_stat_statements** — Анализ частоты и тяжёлых запросов
- **Prometheus** — Сбор и визуализация метрик
- **pgbench** — Имитирование нагрузки
- **Grafana** — Построение дашбордов

🔮 Шаг 10:

Миссия 10

Сопоставьте риски с их последствиями при планировании нагрузки.

Подсказка: Каждый риск ведёт к конкретной боли.

Ответ:

- **Отсутствие резервного планирования** — Простой системы при сбоях
- **Недооценка роста пользователей** — Перегрузка сервера
- **Нет мониторинга** — Слепота перед проблемами
- **Нет автоматизации** — Долгие ручные реакции

🔮 Шаг 11:

Миссия 11

Сопоставьте стратегию с ожидаемым результатом.

Подсказка: Подумайте, что даёт каждый инструмент.

Ответ:

- **Оптимизация запросов** — Снижение времени ответа
- **Индексирование** — Ускорение выборки
- **Кэширование** — Снижение нагрузки на сервер
- **Архивирование старых данных** — Освобождение места

Шаг 12:

Миссия 12

Расставьте шаги долгосрочного планирования нагрузки в правильном порядке.

Подсказка: Сначала измерения, потом прогнозы, затем архитектура.

Ответ:

- Сбор текущих метрик
- Построение прогнозов
- Выбор архитектурного решения
- Тестирование сценариев нагрузки

Шаг 13:

Миссия 13

Упорядочите действия администратора при обнаружении перегрузки.

Подсказка: Сначала анализ, потом временное решение, затем постоянное.

Ответ:

- Проверка метрик
- Применение временных ограничений
- Оптимизация долгих запросов
- Пересмотр архитектуры

Шаг 14:

Миссия 14

Расположите шаги по внедрению мониторинга для прогнозирования роста.

Подсказка: Сначала ставим систему, потом собираем данные, затем строим отчёты.

Ответ:

- Выбор инструмента мониторинга
- Сбор метрик
- Настройка дашбордов
- Анализ тенденций

Шаг 15:

Миссия 15

Расположите шаги оптимизации запросов по мере роста нагрузки.

Подсказка: Начинаем с анализа, заканчиваем изменением структуры данных.

Ответ:

- Сбор медленных запросов
- Добавление индексов
- Переписывание SQL
- Нормализация или денормализация таблиц

Шаг 16:

Миссия 16

Расставьте шаги в правильном порядке при подготовке к нагрузочному тестированию.

Подсказка: Нельзя тестировать без сценариев и контроля.

Ответ:

- Определение целей тестирования
- Составление сценариев
- Запуск теста
- Анализ результатов

Урок 2 - Резервирование и отказоустойчивость

Шаг 01:

Миссия 1

Разработчик Даша настраивает сервер PostgreSQL для стартапа. Она задумалась о резервном копировании: что нужно обязательно предусмотреть для минимизации потерь при сбое?

Подсказка: Важно не только сделать копию, но и уметь восстановить данные.

Ответ:

- **Настроить регулярное тестируемое резервное копирование**
- Ограничиться увеличением памяти сервера
- Просто хранить копию базы на том же диске
- Ничего не делать, ведь PostgreSQL и так надёжен

Шаг 02:

Миссия 2

Андрей заметил: база растёт, а бэкапы стали занимать слишком много места. Какой вариант стоит рассмотреть?

Подсказка: Подумайте о методах, которые не копируют весь объём каждый раз.

Ответ:

- **Использовать инкрементальные или дифференциальные бэкапы**
- Удалять старые данные без копирования
- Хранить только схемы без данных
- Просто увеличить размер диска без изменений стратегии

Шаг 03:

Миссия 3

Компания внедряет кластер PostgreSQL. Какой подход чаще всего используют для повышения отказоустойчивости?

Подсказка: Речь о механизме, где один узел ведёт, а другие повторяют за ним.

Ответ:

- **Настройка репликации Master-Slave**
- Ежедневный ручной экспорт CSV
- Запуск второго PostgreSQL без синхронизации
- Просто установка RAID-массива

Шаг 04:

Миссия 4

У Ирины база работает в облаке. Какой тип резервирования обеспечивает наименьшее время простоя?

Подсказка: Подумайте о механизме, когда резервный сервер всегда готов принять нагрузку.

Ответ:

- **Горячее резервирование (hot standby)**
- Холодное резервирование (cold backup)
- Еженедельные дампы
- Хранение логов в отдельной папке

Шаг 05:

Миссия 5

Виктор настроил горячее резервирование, но переживает за задержки. Что может стать причиной лагов между основным и резервным сервером?

Подсказка: Пропускная способность сети и обработка логов играют роль.

Ответ:

- **Медленная сеть и недостаток ресурсов на реплике**
- Слишком много индексов в базе
- Чрезмерное количество JOIN-запросов
- Использование SSD вместо HDD

Шаг 06:

Миссия 6

Аркадий тестирует отказоустойчивость. Как правильно смоделировать сбой для проверки кластера?

Подсказка: Нужно проверить переключение без ручных манипуляций с данными.

Ответ:

- **Искусственно отключить основной сервер**
- Удалить все таблицы в базе
- Остановить pg_dump во время работы
- Запустить бесконечный цикл SELECT

Шаг 07:

Миссия 7

Сопоставьте типы резервирования с их характеристиками.

Подсказка: Вспомните разницу между тёплым, горячим и холодным резервом.

Ответ:

- **Холодное резервирование** — Резерв запускается вручную, требуется больше времени на восстановление
- **Тёплое резервирование** — Сервер подготовлен, но данные не синхронизируются мгновенно
- **Горячее резервирование** — Реплика постоянно синхронизируется и готова принять нагрузку
- **Инкрементальный бэкап** — Хранит только изменения с последнего сохранения

🔮 Шаг 08:

Миссия 8

Сопоставьте типы отказоустойчивости с их описаниями.

Подсказка: Некоторые механизмы работают на уровне диска, другие — на уровне базы.

Ответ:

- **RAID** — Аппаратная или программная защита на уровне дисков
- **Стриминг-репликация** — Передача WAL-журналов для синхронизации базы
- **Фейловер** — Автоматическое переключение на резервный сервер
- **PgBackRest** — Инструмент для резервного копирования и восстановления

🔮 Шаг 09:

Миссия 9

Сопоставьте инструменты с их назначением.

Подсказка: Один отвечает за мониторинг, другой за бэкап, третий за кластеризацию.

Ответ:

- **Patroni** — Автоматизация управления кластером PostgreSQL
- **PgBouncer** — Лёгкий пул соединений для PostgreSQL
- **PgBackRest** — Надёжные резервные копии с поддержкой инкрементальных бэкапов
- **Zabbix** — Система мониторинга производительности

🔮 Шаг 10:

Миссия 10

Сопоставьте сценарии с подходящим решением.

Подсказка: Подумайте о разнице между короткими сбоями и катастрофическими потерями.

Ответ:

- **Короткое отключение электричества** — Использовать UPS для поддержания работы
- **Повреждение диска** — Использовать RAID или заменить диск с восстановлением из реплики
- **Полный сбой дата-центра** — Организовать географически распределённую репликацию
- **Случайное удаление таблицы** — Восстановление из последнего резервного бэкапа

🔮 Шаг 11:

Миссия 11

Сопоставьте понятие с примером применения.

Подсказка: Каждый термин связан с определённой стратегией защиты данных.

Ответ:

- **Point-In-Time Recovery** — Восстановление базы на конкретный момент времени
- **Hot Standby** — Реплика работает только для чтения, доступна сразу
- **Logical Replication** — Передача изменений на уровне SQL-запросов
- **Archive Mode** — Хранение всех WAL-журналов для восстановления

🔮 Шаг 12:

Миссия 12

Расположите шаги настройки репликации PostgreSQL в правильном порядке.

Подсказка: Сначала нужно подготовить основную базу, затем резервную, потом запустить процесс синхронизации.

Ответ:

- Сконфигурировать primary сервер
- Создать базовую копию данных для реплики
- Настроить параметры подключения standby
- Запустить потоковую репликацию

 Шаг 13:

Миссия 13

Расположите действия администратора при восстановлении базы из резервной копии.

Подсказка: Подумайте: сначала нужно подготовить среду, потом восстановить данные, затем проверить.

Ответ:

- Подготовить чистую среду
- Запустить команду восстановления из бэкапа
- Применить WAL-журналы при необходимости
- Проверить целостность и доступность данных

 Шаг 14:

Миссия 14

Расположите шаги при планировании географически распределённого кластера PostgreSQL.

Подсказка: Начинаем с анализа потребностей, заканчиваем тестированием отказоустойчивости.

Ответ:

- Анализ требований и сценариев отказа
- Выбор дата-центров и сетевых каналов
- Настройка репликации между площадками
- Тестирование переключения при сбое

 Шаг 15:

Миссия 15

Расположите шаги по проверке отказоустойчивости в тестовой среде.

Подсказка: Последовательность: подготовка, имитация сбоя, переключение, анализ результатов.

Ответ:

- Подготовка стенда с резервными серверами
- Имитация сбоя основного сервера
- Автоматическое или ручное переключение
- Анализ времени простоя и логов

Модуль 10 - Экспертные подходы и новые технологии

Урок 1 - Использование Citus для масштабируемости

Шаг 01:

Миссия 1

Мария разрабатывает распределённую базу данных с помощью Citus. Какое основное преимущество Citus перед обычным PostgreSQL при больших объёмах данных?

Подсказка: Подумайте о горизонтальном масштабировании и распределении нагрузки.

Ответ:

- **Масштабирование данных по нескольким узлам**
- Ускорение одиночного SELECT на одном сервере
- Снижение объёма памяти на одном узле
- Замена SQL на NoSQL запросы

Шаг 02:

Миссия 2

Дмитрий готовит сервер Citus для аналитического проекта. Какой подход к шардингу таблиц поможет равномерно распределить данные?

Подсказка: Подумайте о том, чтобы каждая часть данных оказалась на отдельном узле.

Ответ:

- **Шардинг по ключу, который часто используется в запросах**
- Шардинг по случайной строке
- Хранение всех данных на одном узле
- Шардинг по типу данных

Шаг 03:

Миссия 3

Екатерина анализирует нагрузку на Citus-кластер. Какой тип запросов обычно требует согласованного доступа к нескольким узлам?

Подсказка: Подумайте о запросах, которые объединяют данные с разных узлов.

Ответ:

- **Запросы с JOIN по шардам на нескольких узлах**
- Простые SELECT на одной таблице шардированной на один узел
- Локальные индексные запросы
- Запросы без WHERE условия

Шаг 04:

Миссия 4

Сергей планирует масштабировать PostgreSQL с помощью Citus. Какой режим кластера лучше выбрать для OLAP-аналитики?

Подсказка: Подумайте о параллельной обработке больших объёмов данных.

Ответ:

- **Режим распределённого кластера (distributed)**
- Локальный режим на одном сервере

- Режим только для транзакций (OLTP)
- Режим без репликации

🔗 Шаг 05:

Миссия 5

Ольга замечает, что некоторые запросы в Citus выполняются медленно. Что следует проверить в первую очередь?

Подсказка: Важно убедиться, что запросы используют правильный шард-кей и индексы.

Ответ:

- **Использование распределённых индексов и ключа шардинга**
- Количество соединений на одном узле
- Версию PostgreSQL на каждом узле
- Размер WAL-файлов

🔗 Шаг 06:

Миссия 6

Сопоставьте ключевые концепции Citus с их описаниями.

Подсказка: Некоторые термины связаны с распределением данных, другие с обработкой запросов.

Ответ:

- **Shard** — Часть таблицы, распределённая на отдельный узел
- **Worker Node** — Сервер, на котором хранятся шарды
- **Coordinator Node** — Узел, который принимает запросы и распределяет их по воркерам
- **Distributed Query** — Запрос, который выполняется одновременно на нескольких узлах

🔗 Шаг 07:

Миссия 7

Сопоставьте типы таблиц Citus с их особенностями.

Подсказка: Разные таблицы имеют разные возможности для распределения данных.

Ответ:

- **Distributed Table** — Таблица, данные которой распределяются по шард-узлам
- **Reference Table** — Маленькая таблица, копируемая на все узлы
- **Local Table** — Таблица, которая хранится только на одном узле
- **Foreign Table** — Таблица, которая подключается с внешнего источника

🔗 Шаг 08:

Миссия 8

Сопоставьте понятие Citus с примером использования.

Подсказка: Один инструмент помогает ускорить аналитические запросы, другой уменьшает нагрузку на координационный узел.

Ответ:

- **Distributed Tables** — Для хранения больших объёмов данных на нескольких узлах
- **Reference Tables** — Для справочной информации, доступной на всех узлах
- **Worker Nodes** — Для выполнения запросов на данных шардов
- **Coordinator Node** — Для распределения запросов и агрегации результатов

Шаг 09:

Миссия 9

Сопоставьте проблемы масштабирования с их решениями в Citus.

Подсказка: Подумайте, какие типы таблиц и индексов помогают равномерно распределить нагрузку.

Ответ:

- **Горячая точка данных на одном узле** — Перераспределение данных с помощью hash-шардинга
- **Большие JOIN между шардами** — Использование reference таблиц для справочной информации
- **Запросы с агрегацией на многих узлах** — Использование distributed query для параллельной обработки
- **Нехватка ресурсов на одном узле** — Добавление дополнительных воркер-нод

Шаг 10:

Миссия 10

Сопоставьте термины Citus с их эффективностью.

Подсказка: Подумайте, что ускоряет распределённые операции и что снижает накладные расходы.

Ответ:

- **Distributed Index** — Ускоряет поиск по распределённым таблицам
- **Local Index** — Используется только на конкретном узле
- **Reference Table** — Снижает количество JOIN между шардами
- **Distributed Table** — Позволяет обрабатывать большие объёмы данных параллельно

Шаг 11:

Миссия 11

Расположите шаги настройки распределённой таблицы Citus в правильном порядке.

Подсказка: Сначала создаём таблицу, затем выбираем ключ шардинга, потом распределяем по воркерам.

Ответ:

- Создать таблицу на координационном узле
- Выбрать столбец для шардинга
- Распределить таблицу по воркер-нодам
- Проверить доступность данных на всех узлах

Шаг 12:

Миссия 12

Расположите действия администратора для оптимизации запросов в Citus.

Подсказка: Сначала анализируем запрос, потом распределяем данные и индексируем.

Ответ:

- Проанализировать медленные запросы
- Определить подходящий ключ шардинга
- Добавить распределённые и локальные индексы
- Перепроверить производительность после изменений

Шаг 13:

Миссия 13

Расположите этапы внедрения Citus в существующую PostgreSQL-базу.

Подсказка: Начинаем с анализа, заканчиваем тестированием производительности.

Ответ:

- Оценить потребности в масштабируемости
- Выбрать таблицы для распределения
- Настроить координационный и воркер-узлы
- Провести тесты производительности и корректности

Шаг 14:

Миссия 14

Расположите шаги при обновлении данных на распределённой таблице Citus.

Подсказка: Подумайте о том, как изменения применяются на шард-узлах.

Ответ:

- Определить строки для обновления
- Синхронизировать изменения с координационным узлом
- Применить изменения на соответствующих воркерах
- Проверить целостность и согласованность данных

Шаг 15:

Миссия 15

Расположите шаги при настройке масштабируемого кластера Citus для OLAP-аналитики.

Подсказка: Сначала планируем архитектуру, затем распределяем данные, потом проверяем производительность.

Ответ:

- Спланировать архитектуру кластера
- Настроить координационный и воркер-узлы
- Распределить таблицы по шард-узлам
- Провести нагрузочное тестирование

Шаг 16:

Миссия 16

Расположите шаги при восстановлении распределённой таблицы Citus после сбоя.

Подсказка: Начинаем с анализа, затем восстановление шардов, синхронизация и проверка данных.

Ответ:

- Проанализировать степень повреждений
- Восстановить данные на воркер-нодах
- Синхронизировать координационный узел
- Проверить согласованность и целостность данных

Урок 2 - Как управлять большими данными с PostgreSQL

Шаг 01:

Миссия 1

Иван работает с огромной базой PostgreSQL для аналитики. Какой подход поможет ему эффективно управлять большими объёмами данных?

Подсказка: Подумайте о способах разделения данных и ускорения запросов.

Ответ:

- **Использование партиционирования таблиц**
- Перенос всех данных в один большой массив
- Отказ от индексов и ограничений
- Удаление всех исторических данных

Шаг 02:

Миссия 2

София анализирует таблицу с миллиардами записей. Какой метод индексирования поможет ускорить фильтрацию по дате?

Подсказка: Подумайте о типичных сценариях временных рядов и аналитики.

Ответ:

- **Создание BRIN-индекса по столбцу даты**
- Создание уникального индекса по всем столбцам
- Удаление всех индексов
- Индексирование текстового поля описания

Шаг 03:

Миссия 3

Алексей хочет уменьшить время загрузки исторических данных в PostgreSQL. Что ему поможет?

Подсказка: Подумайте о хранении данных по сегментам и упрощении сканирования.

Ответ:

- **Партиционирование таблиц по временным интервалам**
- Объединение всех данных в одну таблицу
- Соккрытие таблиц в отдельных схемах без индексов
- Удаление исторических данных старше года

Шаг 04:

Миссия 4

Наталья обрабатывает большие объёмы логов PostgreSQL. Какой подход ускорит выборку только нужных записей?

Подсказка: Подумайте о создании индексных структур и выборочном хранении данных.

Ответ:

- **Создание частичных индексов по часто используемым фильтрам**
- Создание индекса на все столбцы таблицы
- Отключение всех индексов для ускорения вставки
- Использование временных таблиц без индексов

🔮 Шаг 05:

Миссия 5

Михаил строит OLAP-отчеты на больших данных. Какой инструмент PostgreSQL поможет ускорить агрегации?

Подсказка: Подумайте о структурировании данных для быстрых вычислений.

Ответ:

- **Материализованные представления с периодическим обновлением**
- Создание обычных представлений без индексации
- Хранение всех данных в одной большой таблице
- Удаление агрегатов и пересчет каждый раз с нуля

🔮 Шаг 06:

Миссия 6

Сопоставьте методы управления большими данными с их преимуществами.

Подсказка: Каждый метод решает конкретную проблему хранения или ускорения запросов.

Ответ:

- **Партиционирование таблиц** — Ускоряет выборку и облегчает управление большими таблицами
- **BRIN-индексы** — Снижают объем памяти для индексирования и ускоряют фильтры по диапазонам
- **Материализованные представления** — Позволяют быстро получать агрегированные данные
- **Частичные индексы** — Оптимизируют запросы с предикатами на подмножество данных

🔮 Шаг 07:

Миссия 7

Сопоставьте виды индексов PostgreSQL с их применением для больших данных.

Подсказка: Разные индексы подходят для разных типов запросов.

Ответ:

- **B-Tree** — Для точного поиска и сортировки
- **GIN** — Для поиска по массивам или JSONB
- **BRIN** — Для быстрого фильтра по большим диапазонам
- **Hash** — Для равномерного поиска по ключу

🔮 Шаг 08:

Миссия 8

Сопоставьте типы таблиц PostgreSQL с их ролью в больших данных.

Подсказка: Подумайте о том, какие таблицы ускоряют вставку, а какие — чтение.

Ответ:

- **Основная таблица** — Хранит полные данные и поддерживает целостность
- **Временная таблица** — Используется для промежуточных вычислений и быстрой вставки
- **Материализованное представление** — Для быстрого получения агрегированных данных
- **Партиционированная таблица** — Разделяет данные по диапазонам или ключам для ускорения выборки

Шаг 09:

Миссия 9

Расположите шаги оптимизации работы с большими таблицами в правильном порядке:

1. Разделение таблицы на партиции по ключу или диапазону
2. Создание соответствующих индексов (BRIN, GIN)
3. Настройка конфигурации PostgreSQL для больших данных
4. Мониторинг запросов и корректировка схемы при необходимости

Подсказка: Сначала структурируем данные, потом индексируем, затем настраиваем систему и проверяем эффективность.

Ответ:

- Разделение таблицы на партиции по ключу или диапазону
- Создание соответствующих индексов (BRIN, GIN)
- Настройка конфигурации PostgreSQL для больших данных
- Мониторинг запросов и корректировка схемы при необходимости

Шаг 10:

Миссия 10

Расположите этапы работы с материализованными представлениями для ускорения отчётов:

1. Создание представления с агрегатами
2. Добавление индексов для часто используемых фильтров
3. Настройка периодического обновления
4. Использование представления в запросах аналитики

Подсказка: Сначала создаём представление, затем индексируем, потом обновляем и используем.

Ответ:

- Создание представления с агрегатами
- Добавление индексов для часто используемых фильтров
- Настройка периодического обновления
- Использование представления в запросах аналитики

Шаг 11:

Миссия 11

Расположите шаги подготовки к хранению больших JSONB-документов:

1. Создание таблицы с полем JSONB
2. Определение индексов GIN для быстрого поиска
3. Разделение данных по логическим категориям
4. Мониторинг производительности и корректировка схемы

Подсказка: Сначала создаём структуру, затем индексируем, делим и проверяем эффективность.

Ответ:

- Создание таблицы с полем JSONB
- Определение индексов GIN для быстрого поиска
- Разделение данных по логическим категориям
- Мониторинг производительности и корректировка схемы

Шаг 12:

Миссия 12

Расположите шаги обработки временных таблиц для промежуточных вычислений:

1. Создание временной таблицы
2. Заполнение её данными для расчётов
3. Выполнение необходимых вычислений
4. Удаление временной таблицы после использования

Подсказка: Временные таблицы используются только на время вычислений, их нужно правильно создавать и очищать.

Ответ:

- Создание временной таблицы
- Заполнение её данными для расчётов
- Выполнение необходимых вычислений
- Удаление временной таблицы после использования

Шаг 13:

Миссия 13

Расположите этапы партиционирования таблиц по диапазону дат:

1. Определение диапазонов дат для партиций
2. Создание партиций с соответствующими ограничениями
3. Перенос существующих данных в партиции
4. Обновление запросов для работы с партиционированной таблицей

Подсказка: Сначала планируем диапазоны, затем создаём партиции, переносим данные и настраиваем запросы.

Ответ:

- Определение диапазонов дат для партиций
- Создание партиций с соответствующими ограничениями
- Перенос существующих данных в партиции
- Обновление запросов для работы с партиционированной таблицей

Шаг 14:

Миссия 14

Расположите шаги работы с агрегированными отчётами в PostgreSQL:

1. Создание агрегирующих функций и представлений
2. Добавление индексов на ключи фильтрации
3. Периодическое обновление представлений
4. Использование агрегатов в аналитических запросах

Подсказка: Сначала создаём функции и представления, затем индексируем, обновляем и используем.

Ответ:

- Создание агрегирующих функций и представлений
- Добавление индексов на ключи фильтрации
- Периодическое обновление представлений

- Использование агрегатов в аналитических запросах

Шаг 15:

Миссия 15

Расположите этапы оптимизации хранения больших массивов данных JSONB:

1. Выделение логических блоков данных
2. Создание таблиц с полями JSONB
3. Создание GIN-индексов для быстрого поиска
4. Мониторинг использования и корректировка структуры

Подсказка: Структурируем данные, создаём таблицы, индексируем и проверяем производительность.

Ответ:

- Выделение логических блоков данных
- Создание таблиц с полями JSONB
- Создание GIN-индексов для быстрого поиска
- Мониторинг использования и корректировка структуры

Шаг 16:

Миссия 16

Расположите шаги комплексного подхода к обработке больших данных в PostgreSQL:

1. Разделение таблиц на партиции
2. Создание индексов для ускорения выборки
3. Использование материализованных представлений для агрегатов
4. Мониторинг и оптимизация запросов

Подсказка: Сначала структурируем данные, затем индексируем, создаём агрегаты и следим за эффективностью.

Ответ:

- Разделение таблиц на партиции
- Создание индексов для ускорения выборки
- Использование материализованных представлений для агрегатов
- Мониторинг и оптимизация запросов